

**Optimización de la gestión de memoria en simuladores cuánticos mediante la
compresión de datos sin pérdida de precisión**

Daniel Adrián González Buendía

Javier Andres Sarmiento Salazar

Trabajo de grado para optar el título de Ingeniero de Sistemas

Director

Luis Alberto Nuñez Martínez

PhD en Ciencias de la Computación

Codirector

Gilberto Javier Díaz Toro

PhD en Ciencias de la Computación

Universidad Industrial de Santander

Facultad De Ingenierías Fisicomecánicas

Escuela De Ingeniería De Sistemas e Informática

Pregrado en Ingeniería de Sistemas e Informática Bucaramanga

2026

Dedicatoria

Este proyecto va dedicado a todas las personas que me apoyaron y aportaron a mi desarrollo como persona, y como profesional. A todas las personas que quiero.

A mis padres.

A mi hermana.

A mis amigos.

A Daniel.

Contenido

Introducción	4
1. Planteamiento del problema	5
2. Objetivos	7
2.1. Objetivo General	7
2.2. Objetivos Específicos	7
3. Marco Teórico	8
3.1. Representación del estado cuántico	8
3.2. Evolución unitaria y simulación tipo Schrödinger	9
3.3. Patrones de acceso inducidos por las compuertas cuánticas	9
3.4. Alternativas para reducir memoria en simulación cuántica clásica	10
3.5. Compresión de datos científicos en punto flotante	11
3.6. Compresión por bloques y jerarquía de memoria	12
3.7. Exactitud numérica y métricas de comparación	13
4. Estado del Arte	15
4.1. Simuladores Cuánticos	15
4.2. Compresión de Memoria con Pérdida de Precisión	16
4.3. Compresión de Memoria sin Pérdida de Precisión	17
4.4. Algoritmos de compresión sin pérdida de precisión	19
4.4.1. FPC (Burtscher & Ratanaworabhan)	19
4.4.2. FPZIP (Lindstrom & Isenburg)	20
4.4.3. Filtro Bitshuffle con LZ4 (Masui et al.)	21
4.4.4. Transformación de Datos Tipados (TDT)	22
4.5. Otros Métodos de Optimización de Memoria	23

5. Metodología de la Investigación	24
5.1. Diseño	25
5.2. Implementación y optimización del modelo	26
5.3. Evaluación experimental	26
6. Simuladores cuánticos	27
6.1. Motivación y necesidad de un simulador base	27
6.2. Criterios de selección	28
6.3. Alternativas de simuladores evaluadas	30
6.4. Matriz de evaluación ponderada	31
6.5. Selección del simulador: TMFQSfullstate	32
7. Selección de una biblioteca de compresión sin pérdida para integración en TMFQSfullstate	33
7.1. Motivación y contexto de selección	33
7.2. Criterios de selección	33
7.3. Alternativas de compresión sin pérdida consideradas	35
7.4. Matriz de evaluación ponderada	36
7.5. Selección del algoritmo recomendado	36
8. Patrones en vectores de estado de algoritmos cuánticos relevantes para compresión	37
8.1. Introducción	37
8.2. Búsqueda de Grover: amplitudes uniformes y estados de baja entropía	37
8.3. Transformada Cuántica de Fourier: patrones de fase y baja redundancia	38
8.4. Algoritmo de Shor: periodicidad, dispersión y picos	39
8.5. Deutsch–Jozsa y Simon: superposiciones estructuradas y esparsidad	39
8.6. Circuitos variacionales y circuitos aleatorios: entre regularidad y alta entropía	40
8.7. Implicaciones para compresión sin pérdida en simulación de estado completo	40

9. Diseño de simulación para compresión sin pérdida en simuladores tipo Schrödinger	41
9.1. Objetivos del diseño	42
9.2. Representación por bloques del vector de estado	42
9.3. Descompresión selectiva y estrategia de acceso	43
9.4. Caché LRU de bloques descomprimidos	44
9.5. Flujo de compresión y descompresión	45
9.6. Ventajas, parámetros y limitaciones del diseño	46
10.Desarrollo e implementación del backend basado en Blosc	47
10.1. Contexto de implementación	47
10.2. Backend de almacenamiento comprimido basado en Blosc	48
10.3. Disposición en bloques y mapeo del vector de estado	48
10.4. Caché LRU de bloques descomprimidos	49
10.5. Estrategia de acceso y actualización local	49
11.Evaluación experimental y resultados	51
11.1. Diseño experimental	51
11.2. Resultados para Grover	52
11.3. Resultados para QFT	54
11.4. Comparación global de estrategias	55
Referencias	56

Lista de figuras

1.	Metodología de investigación	25
2.	Arquitectura abstracta para integración de compresión sin pérdida en simuladores tipo Schrödinger	41
3.	Particionamiento abstracto del vector de estado en bloques independientes .	43
4.	Esquema conceptual de la caché LRU de bloques descomprimidos	45
5.	Flujo abstracto de acceso y actualización de bloques comprimidos	46
6.	Arquitectura del desarrollo implementado para el backend basado en Blosc .	47
7.	Mapeo entre estados base y bloques físicos en el backend comprimido	49
8.	Secuencia operativa del backend basado en Blosc	50
9.	Esquema general de la campaña experimental	52
10.	Uso máximo de memoria para Grover	53
11.	Tiempo de ejecución para Grover	53
12.	Uso máximo de memoria para QFT	54
13.	Error máximo absoluto para QFT con compresión con pérdida	54
14.	Síntesis comparativa de resultados experimentales	55

Lista de tablas

1.	Criterios usados para la selección del simulador cuántico	29
2.	Evaluación de alternativas (escala 1–5) y total ponderado	31
3.	Criterios para la selección de bibliotecas de compresión sin pérdida	34
4.	Evaluación de bibliotecas de compresión sin pérdida (escala 1–5) y total ponderado	36
5.	Configuraciones experimentales evaluadas	52
6.	Resultados promedio para Grover con 30 qubits	53
7.	Resultados promedio para QFT con 30 qubits	54
8.	Resumen comparativo de las configuraciones más favorables	55

Lista de apéndices

Glosario

HPC: Computación de alto rendimiento (High Performance Computing)

LRU: Least Recently Used, política de reemplazo de caché basada en el uso más reciente

QFT: Transformada Cuántica de Fourier (Quantum Fourier Transform)

Resumen

Título: Optimización de la gestión de memoria en simuladores cuánticos mediante la compresión de datos sin pérdida de precisión¹

Autores: Daniel Adrián González Buendía
Javier Andres Sarmiento Salazar²

Palabras clave: Computación cuántica, Simulación cuántica, Gestión de memoria, Compresión de datos, Computación de Alto Rendimiento (HPC)

Descripción:

La simulación de sistemas cuánticos en hardware clásico enfrenta una limitación severa debido al crecimiento exponencial en la memoria requerida. A medida que aumenta el número de qubits, la cantidad de datos a almacenar supera rápidamente las capacidades de las computadoras convencionales y supercomputadoras. Esta restricción impide la exploración de algoritmos cuánticos avanzados y dificulta la validación de experimentos en entornos clásicos antes de su ejecución en hardware cuántico. Por ello, es fundamental desarrollar estrategias eficientes de gestión de memoria que permitan extender el alcance de estas simulaciones sin comprometer la exactitud numérica de los cálculos. A pesar de los avances en simuladores cuánticos y técnicas de optimización, la gestión de memoria sigue siendo un cuello de botella significativo. En particular, muchas soluciones actuales sacrifican exactitud a cambio de eficiencia, lo que limita su aplicabilidad en contextos donde la precisión es fundamental. Por ello, es necesario explorar estrategias que reduzcan el consumo de memoria sin comprometer la corrección de los resultados. Este trabajo propone el diseño e implementación de una estrategia de compresión de memoria sin pérdida de precisión en simuladores cuánticos, con el objetivo de encontrar un balance óptimo entre uso de memoria, rendimiento y exactitud numérica. Al final, se espera evaluar la viabilidad de la propuesta y compararla con otros enfoques existentes mediante métricas de memoria, tiempo y error respecto a una ejecución

¹Trabajo de Investigación

²Facultad De Ingenierías Fisicomecánicas. Escuela De Ingeniería De Sistemas e Informática.
Director: PhD. Luis Alberto Nuñez Martínez, Codirector: PhD. Gilberto Javier Díaz Toro

densa de referencia.

Abstract

Title: Optimization of memory management in quantum simulators through data compression without loss of precision³

Authors: Daniel Adrián González Buendía
Javier Andres Sarmiento Salazar⁴

Key words: Quantum Computing, Quantum Simulation, Memory Management, Data Compression, High-Performance Computing (HPC)

Description:

The simulation of quantum systems on classical hardware faces a severe limitation due to the exponential growth in memory requirements. As the number of qubits increases, the amount of data to be stored quickly exceeds the capabilities of conventional computers and supercomputers. This restriction hinders the exploration of advanced quantum algorithms and complicates the validation of experiments in classical environments before their execution on quantum hardware. Therefore, it is crucial to develop efficient memory management strategies that extend the scope of these simulations without compromising numerical exactness. Despite advances in quantum simulators and optimization techniques, memory management remains a significant bottleneck. In particular, many current solutions trade accuracy for efficiency, limiting their applicability in contexts where exact results are crucial. Therefore, it is necessary to explore strategies that reduce memory consumption without compromising the correctness of the simulation. This work proposes the design and implementation of a lossless memory compression strategy for quantum simulators, aiming to find an optimal balance between memory usage, performance, and numerical exactness. Ultimately, the feasibility of the proposal will be evaluated and compared with existing approaches using memory, runtime, and error metrics against a dense reference execution.

³Research Work

⁴Faculty of Physics-Mechanics Engineering. School of Systems Engineering and Computer Science.
Advisor: PhD. Luis Alberto Nuñez Martínez, Co-Advisor: PhD. Gilberto Javier Díaz Toro

Introducción

La simulación cuántica en computadoras clásicas enfrenta un desafío fundamental debido al crecimiento exponencial del espacio de estados conforme aumenta el número de qubits. Por ejemplo, la representación de un sistema cuántico de 40 qubits requiere aproximadamente $8 \text{ bytes (double)} \times 2 \text{ (real, imag)} \times 2^{40} = 16 \text{ TB}$ de memoria RAM, mientras que para 50 qubits la demanda se dispara a petabytes (PB), superando la capacidad de las computadoras convencionales (Wu y cols., 2019a). Esta limitación de memoria restringe el estudio de algoritmos cuánticos avanzados y su validación en hardware clásico antes de su implementación en procesadores cuánticos (Zhang y cols., 2024a). Ante esta problemática, es necesario desarrollar estrategias eficientes de gestión de memoria que permitan extender el alcance de las simulaciones cuánticas sin comprometer la viabilidad computacional.

Considerando la creciente importancia de la computación cuántica, optimizar la memoria en simuladores cuánticos no solo permite ampliar el rango de algoritmos simulables en hardware clásico, sino que también reduce la dependencia exclusiva de procesadores cuánticos experimentales. Además, una gestión eficiente de memoria podría reducir significativamente los costos computacionales y el tiempo de simulación en supercomputadoras, lo que resulta en un beneficio directo tanto para la investigación en computación cuántica como para aplicaciones prácticas en la industria y la ciencia.

Para abordar este problema, se han explorado diversas estrategias que pueden agruparse en varias familias de métodos. Entre ellas, la compresión de datos ha sido ampliamente estudiada con enfoques como la compresión con pérdida controlada (e.g., SZ, ZFP), que permite reducir la carga de almacenamiento de vectores de estado sin afectar significativamente la fidelidad de los cálculos (Wu, Di, Cappello, y cols., 2018). Otra línea de investigación relevante es la de los métodos de podado de estados cuánticos, los cuales eliminan amplitudes irrelevantes para disminuir el consumo de memoria, aunque con un impacto variable en la precisión del resultado (Song, Li, y Wu, 2023). Asimismo, la computación de alto rendimien-

to (HPC o High Performance Computing) ha demostrado ser una solución clave, empleando distribución de datos entre nodos y optimizaciones en el acceso a memoria para maximizar la eficiencia de la simulación en infraestructuras de supercomputadoras (Díaz, Steffenel, Barrios, y Couturier, 2024).

A pesar de los avances en estas áreas, muchas de estas estrategias introducen errores acumulativos en la simulación, lo que puede comprometer la validez de los resultados. En este contexto, esta investigación propone una estrategia basada en compresión de datos sin pérdida de precisión, con el objetivo de optimizar el almacenamiento de vectores de estado sin alterar las amplitudes cuánticas originales. Para ello, el proyecto se desarrollará en varias etapas. Inicialmente, se llevará a cabo una revisión del estado del arte para identificar las técnicas de compresión más relevantes. Luego, se seleccionará un simulador cuántico adecuado que permita modificaciones en su gestión de memoria para evaluar dichas técnicas. Posteriormente, se diseñará una estrategia para seleccionar e integrar técnicas de compresión sin pérdida de precisión existentes, seguida de su implementación y optimización en el simulador elegido. Finalmente, se realizarán experimentos controlados para evaluar el impacto de la compresión en términos de ahorro de memoria, tiempo de simulación y error absoluto y relativo frente a una ejecución densa de referencia, comparándolo con enfoques tradicionales. En las siguientes secciones, se detallarán cada una de estas etapas y su contribución al objetivo final del proyecto.

1 Planteamiento del problema

La simulación de sistemas cuánticos en computadoras clásicas se enfrenta a una barrera casi insuperable: el crecimiento exponencial de la memoria requerida. Representar un estado cuántico de n qubits implica almacenar 2^n números complejos, lo que rápidamente supera la capacidad de cualquier sistema de memoria convencional. Las técnicas actuales, como la poda de estados, reducen drásticamente el consumo de recursos pero a costa de la precisión, mientras que la compresión con pérdida, aunque ofrece mayores índices de reducción, sacri-

fica irremediablemente la fidelidad de los cálculos. Este dilema –la imposibilidad de simular algoritmos cuánticos complejos sin perder información crítica– constituye el núcleo del problema, amenazando la validez y aplicabilidad de las simulaciones en el desarrollo de nuevas estrategias cuánticas.

Este desafío se agrava al considerar que la simulación precisa es esencial para la validación y desarrollo de algoritmos cuánticos en entornos clásicos. La degradación de la precisión debido a la pérdida de datos no solo impide obtener resultados confiables, sino que también limita la escalabilidad y el rendimiento de los simuladores cuánticos. Por ello, es crucial explorar alternativas que permitan optimizar el uso de recursos computacionales sin comprometer la exactitud de los cálculos, destacando la urgente necesidad de investigar métodos de compresión sin pérdida que ofrezcan un equilibrio entre eficiencia y fidelidad en la simulación cuántica.

Con este planteamiento se proponen los siguientes objetivos para abordar esta problemática.

2 Objetivos

2.1 Objetivo General

- Diseñar e implementar una estrategia eficiente de gestión de memoria en simuladores cuánticos basada en técnicas de compresión sin pérdida de precisión, con el fin de optimizar el uso de recursos computacionales sin comprometer la exactitud numérica de las simulaciones.

2.2 Objetivos Específicos

- Analizar el impacto del uso de herramientas de compresión de datos sin pérdida de precisión en el consumo de memoria y en el error absoluto y relativo de los resultados respecto a una referencia densa.
- Diseñar una estrategia integral de gestión de memoria basada en compresión sin pérdida de precisión, definiendo el algoritmo, los parámetros de compresión-descompresión y los puntos de integración con el simulador cuántico seleccionado.
- Desarrollar e integrar la estrategia diseñada en el simulador elegido, optimizando la sobrecarga computacional para garantizar un ahorro significativo de memoria sin afectar la exactitud numérica de la simulación.
- Implementar dos algoritmos cuánticos distintos en un simulador cuántico para evaluar el uso de memoria, velocidad y precisión con y sin compresión de datos.
- Comparar los resultados obtenidos con estrategias tradicionales y métodos que emplean compresión con pérdida de precisión, identificando ventajas y desventajas de cada enfoque.

3 Marco Teórico

La simulación cuántica clásica de circuitos permite estudiar algoritmos, validar resultados y analizar costos computacionales antes de su ejecución en hardware cuántico. Entre los modelos disponibles, la simulación basada en vectores de estado ocupa un lugar central por su generalidad, aunque dicha generalidad viene acompañada de un crecimiento exponencial en memoria y tiempo. En este contexto, resulta necesario revisar los fundamentos de la representación del estado cuántico, los principales enfoques de simulación clásica, los conceptos de compresión de datos científicos y los criterios de exactitud numérica empleados para evaluar representaciones comprimidas y no comprimidas de un mismo estado.

3.1 Representación del estado cuántico

En el modelo de circuitos cuánticos, el estado de un sistema de n qubits se representa mediante un vector de dimensión 2^n :

$$|\psi\rangle = \sum_{i=0}^{2^n-1} \alpha_i |i\rangle,$$

donde cada amplitud $\alpha_i \in \mathbb{C}$ satisface la condición de normalización

$$\sum_{i=0}^{2^n-1} |\alpha_i|^2 = 1.$$

Esta formulación expresa al sistema como una combinación lineal de estados base del espacio de Hilbert. En términos computacionales, cada amplitud debe almacenarse en memoria clásica cuando se utiliza una representación explícita del vector completo. Si se emplea doble precisión, cada amplitud compleja requiere 16 bytes: 8 para la parte real y 8 para la parte imaginaria. El costo teórico de memoria queda entonces dado por:

$$M(n) = 2^n \times 16 \text{ bytes}.$$

El crecimiento exponencial de $M(n)$ constituye una de las principales limitaciones de la simulación cuántica clásica. Para valores moderados de n , el almacenamiento del vector de estado ya exige cantidades de memoria comparables con las de sistemas de cómputo de alto desempeño. Por esta razón, la representación del estado no solo es un concepto matemático fundamental, sino también el origen del problema computacional tratado en simulación cuántica a gran escala.

3.2 Evolución unitaria y simulación tipo Schrödinger

La evolución de un sistema cuántico aislado se modela mediante operadores unitarios. Si U representa una compuerta cuántica o una secuencia de compuertas, el nuevo estado del sistema se obtiene como

$$|\psi'\rangle = U |\psi\rangle.$$

La simulación tipo Schrödinger consiste en mantener explícitamente el vector de estado y aplicar sobre él las transformaciones unitarias del circuito. Su principal ventaja es la generalidad: cualquier circuito que pueda expresarse en el modelo de compuertas puede, en principio, simularse sin imponer restricciones sobre el grado de entrelazamiento o la forma del estado intermedio. Esta propiedad convierte a la simulación Schrödinger en una base natural para contrastar representaciones alternativas del mismo estado cuántico.

Su principal limitación es que tanto el costo de memoria como una parte importante del costo temporal crecen exponencialmente con el número de qubits. En consecuencia, cualquier estrategia orientada a reducir el uso de memoria debe analizarse sin perder de vista el formalismo de evolución unitaria que define el problema original.

3.3 Patrones de acceso inducidos por las compuertas cuánticas

Aunque el vector de estado contiene 2^n amplitudes, las operaciones elementales de un circuito no siempre requieren tratar todas las entradas de manera arbitraria al mismo tiempo. Una compuerta de un solo qubit actúa sobre pares de amplitudes cuyos índices difieren en

el bit asociado al qubit objetivo. De manera similar, una compuerta de dos qubits actúa sobre grupos de cuatro amplitudes relacionadas por las combinaciones binarias de los qubits involucrados.

Desde la perspectiva algorítmica, esto significa que la actualización del estado presenta patrones estructurados de acceso. La importancia de esta observación reside en el hecho de que el costo práctico de simular un circuito depende no solo del tamaño total del vector, sino también de cómo se distribuyen y reutilizan los accesos sobre sus componentes.

El concepto de localidad resulta entonces relevante. En computación, la localidad temporal describe la tendencia de ciertos datos a reutilizarse en un intervalo corto de tiempo, mientras que la localidad espacial se refiere a accesos cercanos dentro de una misma región de memoria. Ambas nociones son fundamentales para comprender por qué algunas representaciones del estado pueden explotarse con mayor eficiencia que otras.

En simulación tipo Schrödinger, la localidad no implica necesariamente que los elementos afectados se encuentren siempre contiguos en memoria, sino que las compuertas inducen relaciones regulares entre índices. Dichas regularidades permiten razonar sobre patrones repetitivos de lectura y escritura, y por ello resultan importantes cuando más adelante se analizan estrategias de almacenamiento para vectores de gran tamaño.

3.4 Alternativas para reducir memoria en simulación cuántica clásica

La literatura ha propuesto diversos enfoques para reducir el costo espacial de la simulación cuántica. Las redes tensoriales y métodos relacionados aprovechan estructuras favorables del circuito, como bajo entrelazamiento o geometrías particulares, para evitar la materialización explícita del vector completo. De forma análoga, los diagramas de decisión cuánticos explotan redundancias algebraicas y simetrías que permiten representar ciertos estados de manera compacta.

También existen técnicas basadas en partición del circuito, formulaciones tipo caminos de Feynman y esquemas híbridos espacio-tiempo, donde la memoria se reduce al costo de

incrementar el tiempo de cómputo o de restringir la clase de circuitos que pueden tratarse eficientemente. Estas alternativas demuestran que el problema de memoria puede abordarse desde distintos formalismos, pero también evidencian que las ventajas obtenidas suelen depender de propiedades estructurales específicas del circuito o del estado.

Cuando se desea conservar la representación explícita del vector de estado, otra línea de trabajo consiste en estudiar estrategias de almacenamiento más eficientes sobre la misma formulación Schrödinger. En ese caso, el interés teórico deja de estar en sustituir el modelo de simulación y pasa a concentrarse en cómo representar, transformar y recuperar los datos numéricos del estado sin alterar su significado físico.

Esta distinción es importante porque no todas las técnicas de reducción de memoria persiguen el mismo objetivo. Algunas cambian la representación matemática del estado para aprovechar estructura; otras mantienen la misma representación y modifican únicamente la manera en que los datos se guardan, se recuperan o se transfieren en memoria. Diferenciar ambos enfoques ayuda a interpretar con mayor claridad las decisiones presentadas en capítulos posteriores.

3.5 Compresión de datos científicos en punto flotante

La compresión de datos busca reducir el espacio requerido para almacenar o transmitir información. En términos generales, puede clasificarse en dos grandes categorías: compresión sin pérdida y compresión con pérdida. La primera garantiza la reconstrucción exacta de los datos originales tras la descompresión; la segunda acepta una perturbación controlada a cambio de mayores ratios de compresión.

En datos científicos representados en punto flotante, la distinción entre ambos enfoques es especialmente importante. Muchas aplicaciones numéricas toleran errores acotados y, por ello, emplean compresión con pérdida. Sin embargo, cuando se requiere preservar exactamente los valores originales, solo la compresión sin pérdida resulta admisible.

Los arreglos de punto flotante presentan desafíos particulares para la compresión porque

con frecuencia exhiben alta entropía aparente. No obstante, incluso en secuencias numéricas complejas pueden existir regularidades en su representación binaria, por ejemplo en signos, exponentes o patrones a nivel de bytes. Por esta razón, una práctica común en compresión científica consiste en aplicar primero transformaciones reversibles que reorganicen los datos y, posteriormente, utilizar compresores sin pérdida de propósito general o de alta velocidad.

Técnicas como *shuffle* y *bitshuffle* son ejemplos de este principio. Ambas reorganizan los bytes o los planos de bits de un conjunto de valores para hacer más visibles ciertas regularidades al compresor posterior. Este tipo de preconditionamiento es particularmente relevante en arreglos homogéneos de gran tamaño, como los que aparecen en simulación numérica y cómputo científico.

Desde el punto de vista conceptual, estas transformaciones no cambian el significado de los datos, sino únicamente su organización antes de comprimir. Su utilidad radica en que muchos compresores funcionan mejor cuando reciben secuencias con patrones repetitivos explícitos. En consecuencia, una parte importante del problema no depende solo del compresor elegido, sino también de cuán favorable resulta la disposición binaria de los datos para dicho compresor.

3.6 Compresión por bloques y jerarquía de memoria

En el manejo de grandes volúmenes de datos científicos, una estrategia frecuente consiste en particionar los arreglos en bloques independientes. Este principio aparece en técnicas de almacenamiento, procesamiento fuera de núcleo, cachés y bibliotecas de compresión orientadas a alto desempeño. La motivación general es que operar sobre unidades más pequeñas permite equilibrar mejor el costo de acceso, el aprovechamiento de la localidad y el uso de memoria temporal.

La granularidad del bloque introduce un compromiso clásico. Bloques grandes tienden a capturar más redundancia y pueden favorecer mejores ratios de compresión, pero suelen incrementar la latencia de acceso cuando solo una fracción pequeña de los datos es nece-

saria. Bloques pequeños reducen el costo de acceso localizado, aunque pueden degradar el rendimiento del compresor y aumentar el peso relativo del metadato y la administración.

Asociado a esta idea aparece el concepto de jerarquía de memoria. En muchos sistemas de cómputo, una parte de los datos permanece en almacenamiento principal, mientras otra se mantiene temporalmente en niveles de acceso más rápido. Las decisiones sobre reemplazo, reutilización y permanencia de los datos en estos niveles intermedios se relacionan con la noción general de caché. Dentro de este marco, políticas como *Least Recently Used* (LRU) constituyen mecanismos clásicos para explotar localidad temporal cuando el conjunto de datos activo excede la capacidad de memoria inmediata.

La intuición detrás de LRU es sencilla: si un dato fue utilizado recientemente, es razonable suponer que tiene mayor probabilidad de reutilizarse en el corto plazo que otro que lleva más tiempo sin accederse. Por ello, cuando la capacidad disponible es limitada, la política expulsa primero el elemento menos recientemente utilizado. Aunque esta regla no siempre produce la decisión óptima, su fundamento coincide con un principio ampliamente aceptado en sistemas de memoria: muchos programas exhiben conjuntos de trabajo que permanecen activos durante intervalos acotados de ejecución.

En este sentido, el valor teórico de una política como LRU no depende de una implementación particular, sino de su relación con la noción de localidad temporal. Comprender este vínculo facilita interpretar por qué la reutilización reciente de datos puede traducirse en menores costos de acceso y, en términos generales, en una mejor utilización de memoria intermedia.

3.7 Exactitud numérica y métricas de comparación

La evaluación de representaciones comprimidas y no comprimidas de un mismo estado requiere distinguir entre ahorro de memoria, costo temporal y exactitud numérica. En cuanto

a memoria, una métrica habitual es la razón de compresión:

$$CR = \frac{S_{\text{sin comp.}}}{S_{\text{comp.}}},$$

donde $S_{\text{sin comp.}}$ representa el tamaño del estado en su forma no comprimida y $S_{\text{comp.}}$ el tamaño total bajo compresión.

En cuanto a desempeño, resulta natural medir el tiempo total de ejecución y la sobrecarga relativa frente a una simulación equivalente sin compresión:

$$OH = \frac{T_{\text{comp}} - T_{\text{sin comp.}}}{T_{\text{sin comp.}}}.$$

Cuando el objetivo es preservar exactamente las amplitudes originales, la corrección debe formularse en primer lugar como una condición de igualdad exacta respecto a una simulación de referencia sin compresión. En términos ideales, esto significa verificar que

$$\alpha_i^{(\text{ref})} = \alpha_i^{(\text{comp})} \quad \forall i.$$

Esta condición expresa de forma directa el significado de una estrategia sin pérdida: el estado recuperado debe coincidir exactamente con el estado de referencia, no solo aproximarse a él. Cuando la simulación utiliza la misma precisión numérica y el mismo orden de operaciones, esta igualdad puede incluso comprobarse a nivel bit a bit.

Como medida auxiliar de reporte, puede emplearse el error absoluto máximo:

$$e_{\text{abs}} = \max_i \left| \alpha_i^{(\text{ref})} - \alpha_i^{(\text{comp})} \right|.$$

Esta métrica no sustituye el criterio de igualdad exacta, pero resulta útil para resumir numéricamente cualquier discrepancia en un único valor. En un esquema estrictamente sin pérdida se espera que $e_{\text{abs}} = 0$.

4 Estado del Arte

4.1 Simuladores Cuánticos

La simulación clásica de circuitos cuánticos es esencial para el desarrollo y validación de algoritmos cuánticos. Sin embargo, el principal desafío radica en la representación del estado cuántico, cuyo tamaño crece exponencialmente con el número de qubits.

Uno de los primeros enfoques para abordar este problema fue la representación mediante *Matrix Product States* (MPS), propuesta por Vidal (2003). Este método permite representar ciertos estados cuánticos de forma eficiente cuando el entrelazamiento es limitado, reduciendo los requisitos de almacenamiento y cómputo. Su principal ventaja es la escalabilidad en circuitos con baja profundidad, permitiendo simular cientos de qubits. No obstante, su limitación radica en que para sistemas altamente entrelazados el costo computacional crece exponencialmente, restringiendo su aplicabilidad a ciertos algoritmos.

A medida que se buscaban métodos más generales, surgió la simulación tipo *Schrödinger*, que almacena el vector de estado completo y lo actualiza en cada operación cuántica. Este enfoque, empleado en simuladores como QuEST y Qiskit Aer (Li, Kimura, Sato, Yu, y Fujita, 2023), garantiza alta precisión y permite simular cualquier circuito cuántico. Su desventaja principal es la extrema demanda de memoria, la cual crece exponencialmente con el número de qubits, alcanzando 0.5 petabytes para 45 qubits y varios exabytes para 50 qubits (Pednault y cols., 2020).

Para mitigar el problema de memoria, se desarrollaron métodos basados en partición del circuito y caminos de Feynman. Por ejemplo, Google e IBM utilizaron esta técnica para simular circuitos de hasta 56 qubits con profundidad limitada (Chen, Zhang, Huang, Newman, y Shi, 2018). Sin embargo, aunque reducen el almacenamiento requerido, estos métodos incrementan la complejidad computacional, ya que el número de caminos crece exponencialmente con la profundidad del circuito.

4.2 Compresión de Memoria con Pérdida de Precisión

Dado que la representación exacta de estados cuánticos es ineficiente en términos de memoria, se han explorado técnicas de compresión con pérdida de precisión. Estos enfoques sacrifican una pequeña cantidad de fidelidad numérica a cambio de una reducción significativa en el uso de memoria, permitiendo la simulación de circuitos más grandes.

Uno de los primeros enfoques en este ámbito fue el de Wu, Di, Cappello, y cols. (2018); Wu y cols. (2019a), quienes introdujeron la compresión adaptativa con error acotado. Utilizando la biblioteca SZ, lograron reducir el tamaño del vector de estado hasta en un factor de 16, manteniendo fidelidades superiores al 99.9 %. Su principal ventaja es que permite ampliar el número de qubits simulados sin un incremento proporcional en los requisitos de memoria. Sin embargo, la compresión y descompresión recurrente introduce sobrecarga computacional, afectando la eficiencia temporal de la simulación.

En un esfuerzo por maximizar la escala de la simulación cuántica de estado completo, Zhang y cols. (2024a) presentaron *BMQSim*, un marco que extiende *SV-Sim* mediante el uso de **compresores con pérdida de precisión y control de error punto a punto** ejecutados directamente en GPU (por ejemplo, variantes de la familia SZ/cuSZ)(Zhang y cols., 2024b). Su diseño combina:

- Una estrategia de *circuit partitioning* que disminuye la frecuencia de (des)compresión.
- Un *pipeline* solapado que oculta en gran medida el coste de mover y comprimir datos.
- Una gestión de memoria jerárquica (VRAM + SSD mediante GPUDirect Storage) que asigna dinámicamente espacio a los bloques comprimidos.

Con estas técnicas, *BMQSim* reduce el consumo de memoria en más de un orden de magnitud y mantiene fidelidades superiores al 99 % sin penalizar de forma apreciable el tiempo de simulación. Su eficacia, sin embargo, está condicionada a la disponibilidad de hardware acelerado con GPUs de alto rendimiento y unidades NVMe rápidas, lo que puede restringir su adopción en plataformas de menor capacidad computacional.

Más recientemente, Huffman, Pavlichin, y Weissman (2024) exploraron la cuantización de amplitudes como un mecanismo para reducir la precisión numérica sin afectar la fidelidad global del sistema. Demostraron que representaciones con tan solo 7 bits por amplitud permiten mantener una fidelidad superior al 99 %, lo que sugiere que la precisión completa de punto flotante puede ser innecesaria en muchas simulaciones. Aunque esta técnica ofrece un enfoque simple y eficiente para reducir el uso de memoria, su aplicabilidad a circuitos de gran escala aún requiere validación empírica.

Díaz Toro (2025) propone un esquema de **vector de estado comprimido** que emplea compresores con pérdida de precisión—principalmente *ZFP* en modo *fixed-rate*, complementado por *SZ*—para reducir la huella de memoria del simulador. El autor reporta reducciones del **10–20 %** en casos base y, en configuraciones de mayor escala, ahorros que alcanzan hasta un **75 %** de la memoria requerida. *ZFP* opera sobre bloques independientes de 4^d valores, de modo que la (des)compresión de cada bloque se realiza en completo aislamiento del resto; esto permite solapar dichas operaciones con el cómputo principal y amortiguar la sobrecarga introducida. Aunque la compresión incrementa el tiempo de ejecución, el impacto se compensa al transmitir los fragmentos del vector en formato comprimido dentro de una arquitectura distribuida basada en *MPI*, reduciendo el volumen de comunicación y habilitando simulaciones de hasta 30 qubits en los experimentos reportados. Finalmente, el estudio concluye que esta estrategia de *estado completo comprimido distribuido* resulta más fiable y escalable que la *poda de estados de baja probabilidad*, la cual presenta descensos de fidelidad y sobrecarga adicional que la hacen desaconsejable.

4.3 Compresión de Memoria sin Pérdida de Precisión

La simulación cuántica eficiente requiere reducir el consumo de memoria sin comprometer la fidelidad del estado cuántico. Para ello, se han desarrollado técnicas de compresión sin pérdida de precisión que buscan representar estados y operadores cuánticos de manera más compacta sin alterar sus valores.

Uno de los primeros enfoques en esta dirección fue el uso de diagramas de decisión cuánticos (*Quantum Information Decision Diagrams*, QuIDD), introducidos por Viamontes, Markov, y Hayes (2005). Este método permite representar vectores de estado y operadores unitarios aprovechando redundancias estructurales en sus coeficientes, lo que permite una representación más eficiente en memoria. Su ventaja radica en que, para circuitos con alta redundancia, el crecimiento en memoria puede reducirse de exponencial a polinomial. Sin embargo, su eficacia se limita a circuitos con estructura repetitiva, fallando en casos de entrelazamiento complejo.

En un desarrollo posterior, Miller y Thornton (2006) propusieron los *Quantum Multiple-valued Decision Diagrams* (QMDD), los cuales extienden la representación de QuIDD mediante el uso de aristas ponderadas con matrices unitarias. Esto permitió manejar de manera más eficiente circuitos reversibles y operadores cuánticos de mayor tamaño. Aunque estos diagramas optimizan la representación de puertas lógicas y matrices densas, su eficiencia sigue dependiendo de la estructura interna del circuito.

Más recientemente, Jaques y Häner (2021) exploraron la esparsidad del estado cuántico para almacenar solo las amplitudes no nulas, reduciendo significativamente el almacenamiento requerido en algoritmos con exploración limitada del espacio de Hilbert. Su enfoque demostró ser altamente eficiente en problemas de factorización y aritmética cuántica, pero pierde efectividad en circuitos con alta interferencia y entrelazamiento.

En 2023, Vinkhuijzen, Coopmans, Elkouss, Dunjko, y Laarman (2023) introdujeron los *Local Invertible Map Decision Diagrams* (LIMDD), una evolución de los diagramas de decisión que incorpora transformaciones locales para mejorar la representación compacta del estado cuántico. Esta técnica permite manejar estados estabilizadores y ciertas estructuras altamente entrelazadas que eran difíciles de representar con diagramas de decisión tradicionales. No obstante, su aplicabilidad sigue limitada a casos donde las transformaciones locales puedan ser explotadas para reducir la complejidad estructural.

Estos avances en compresión sin pérdida han permitido extender la capacidad de simu-

lación de circuitos cuánticos en hardware clásico. Sin embargo, aún existen limitaciones en la representación de estados arbitrarios, lo que ha llevado a la exploración de otros métodos para optimizar el uso de memoria.

4.4 Algoritmos de compresión sin pérdida de precisión

En simulaciones científicas y cuánticas es común manejar vectores de estado representados como grandes arreglos de números de punto flotante. Cuando estos datos carecen de correlación temporal o patrones estructurales (es decir, presentan alta entropía o *aleatoriedad*), la compresión sin pérdida se vuelve especialmente desafiante. Muchos algoritmos de compresión de punto flotante aprovechan transiciones suaves o redundancias entre valores consecutivos; por ejemplo, técnicas de serie temporal almacenan solo los bits que difieren entre un valor y el siguiente, explotando que a menudo los valores sucesivos comparten prefijos de bits iguales. Sin embargo, en datos altamente aleatorios, estas suposiciones se rompen y tales métodos tienden a no lograr compresión significativa. Por lo tanto, es crucial identificar y seleccionar algoritmos de compresión sin pérdida que sean efectivos para datos de punto flotante con estas características. A continuación, se describen los principales algoritmos que han sido concebidos para este tipo de datos, analizando su funcionamiento, rendimiento y pertinencia para ser implementados y evaluados en el contexto de este trabajo de grado, con el fin de encontrar un balance óptimo entre el ahorro de memoria y la eficiencia computacional.

4.4.1 *FPC (Burtscher & Ratanaworabhan)*

El algoritmo *FPC (Floating Point Compressor)* (Burtscher y Ratanaworabhan, 2009) fue diseñado para la compresión sin pérdida de flujos lineales de datos en doble precisión (64 bits). Internamente, FPC emplea un esquema de *predicción* dual junto con codificación de ceros líderes. Concretamente, mantiene dos predictores (derivados de técnicas FCM/DFCM) almacenados en tablas hash, que generan un valor estimado para el siguiente número en la secuencia. Para cada nuevo valor de entrada, FPC selecciona la predicción que más coincide

en sus bits más significativos con el valor real, y calcula el **XOR** entre el valor real y el predicho. Este **XOR** resalta únicamente los bits diferentes, convirtiendo los bits coincidentes en *cero*. A continuación, el algoritmo cuenta cuántos bytes iniciales del resultado **XOR** son cero (ceros líderes) y codifica esa cantidad en 3 bits, junto con 1 bit adicional que indica cuál predictor fue usado. Estos 4 bits de código, seguidos de los *bytes residuales* no nulos (almacenados sin más codificación), forman la salida comprimida.

Esta estrategia implica que, si el valor predicho es cercano al real, la diferencia tendrá muchos ceros líderes y, por tanto, pocos bytes residuales que almacenar. En escenarios de datos altamente aleatorios (donde las predicciones rara vez aciertan), FPC tiende a obtener pocos ceros líderes y la tasa de compresión se aproxima a 1:1 (sin reducción notable). No obstante, su diseño minimiza la penalización en estos casos: los bloques comprimidos incluyen encabezados mínimos y códigos de apenas 4 bits por valor, garantizando sobrecarga reducida y un rendimiento consistente. Así, aunque FPC no logre compresión sustancial en vectores de estado cuántico aleatorios, ofrece una solución de alta velocidad con mínima sobrecarga, capaz de detectar rápidamente si no hay redundancia aprovechable y manejar dichos datos sin degradar el rendimiento.

4.4.2 **FPZIP** (*Lindstrom & Isenbug*)

FPZIP (Lindstrom y Isenbug, 2006) es un compresor de propósito específico para arreglos de punto flotante. Su enfoque se basa en la *predicción adaptativa* y la *codificación entrópica* eficiente para lograr compresión sin pérdida en múltiples contextos (mallados no estructurados, volúmenes 3D, imágenes, etc.). Internamente, FPZIP aplica esquemas de predicción multi-dimensional (por ejemplo, predictores que aprovechan la continuidad espacial de los datos científicos) y luego comprime únicamente el error de predicción. A diferencia de métodos que cuantizan los flotantes a enteros, FPZIP opera directamente sobre las representaciones IEEE 754, manteniendo exactitud bit a bit.

Su arquitectura soporta predictores ajustables al tipo de datos: por ejemplo, en un campo

volumétrico 3D puede usar un predictor basado en valores vecinos, mientras que en una serie temporal podría usar extrapolación simple. Tras la predicción, FPZIP emplea un modelo de codificación entrópica optimizado para velocidad, que comprime los residuos de manera eficaz. Esta combinación le permite alcanzar tasas de compresión y velocidades competitivas, acelerando la E/S de simulaciones científicas al reducir el volumen de datos sin sacrificar precisión. En contextos de datos altamente aleatorios, donde las predicciones en su mayoría fallan, FPZIP tiende a comportarse como un compresor genérico pero con ventajas sutiles: incluso en ausencia de correlación temporal, puede explotar patrones locales dentro de la representación en coma flotante (p. ej., regularidades en campos de exponentes o signos comunes) que los algoritmos genéricos pasan por alto al tratar los datos solo como bytes crudos. Para vectores de estado cuántico, si bien los amplitudes pueden lucir aleatorios, FPZIP podría aprovechar distribuciones no uniformes de ciertos bits (por ejemplo, exponentes repetidos), logrando compresiones modestas con un rendimiento robusto.

4.4.3 Filtro *Bitshuffle* con LZ4 (Masui et al.)

Una estrategia distinta para afrontar datos de punto flotante sin patrones evidentes es emplear transformaciones previas que revelen posibles regularidades a nivel de bits. *Bitshuffle* (Masui y cols., 2015) es un algoritmo de preprocesamiento sin pérdida que extiende el filtro *byte-shuffle* al nivel de *bit*. En lugar de reordenar solo los bytes de cada número (como hace el filtro *shuffle* de HDF5), Bitshuffle realiza una *transposición bit a bit* de una matriz de valores: considera la representación binaria de N números de m bits (por ejemplo, $m=32$ para flotantes en simple precisión) como una matriz de $N \times m$, y reorganiza la data tomando columnas de bits completas. Tras esta permuta, aplica una compresión LZ (usualmente LZ4) sobre la secuencia resultante.

El fundamento es que, incluso si los valores originales parecen aleatorios, puede haber *correlaciones ocultas* dentro de ciertos planos de bits. Por ejemplo, en datos científicos muchos valores comparten el mismo exponente o presentan bits altos similares, lo que tras Bitshuffle

se manifiesta como secuencias repetitivas (*runs* de ceros o unos) más fáciles de comprimir mediante LZ4. A diferencia de métodos predictivos, Bitshuffle no asume continuidad entre elementos adyacentes; su efectividad radica en convertir cualquier correlación intra-byte (dentro de la representación binaria de cada valor) en redundancias horizontales a lo largo de la secuencia transformada. En datos de alta entropía pura (bits totalmente aleatorios), Bitshuffle no puede crear patrones donde no los hay; con todo, en vectores de estado cuántico puede explotar pequeñas irregularidades: si los qubits están normalizados, muchos amplitudes comparten signos o exponentes comunes, generando columnas de bits constantes que LZ4 comprime muy bien. En la práctica, Bitshuffle+LZ4 logra relaciones de compresión superiores a compresores genéricos en escenarios científicos sin reducción de precisión, a la vez que mantiene velocidades de descompresión del orden de gigabytes por segundo por núcleo.

4.4.4 Transformación de Datos Tipados (TDT)

Una innovación reciente en compresión de flotantes es la *Transformación de Datos Tipados* (TDT) (Jamalidinan y Cheshmi, 2025), concebida para cerrar la brecha entre compresores genéricos y especializados en punto flotante. TDT no es un compresor en sí, sino una etapa de *preprocesamiento* adaptable que reorganiza los bytes de los datos antes de aplicar un compresor convencional (p. ej., Zstd o LZ4). La idea clave es agrupar y separar los bytes según su *nivel de entropía* y patrones estadísticos, en lugar de tratarlos uniformemente. En una representación IEEE 754, distintos campos (signo, exponente, mantisa) pueden poseer distribuciones diferentes; por ejemplo, en muchas series científicas los bytes de exponente tienden a repetirse más que los bytes de mantisa. TDT analiza el conjunto de datos (en bloques) extrayendo características estadísticas de cada posición de byte (entropía media, desviación, frecuencias de valores) y aplica *clustering* para decidir cuáles bytes se comportan de forma similar. Luego *reordena* la matriz de datos colocando juntos los bytes de comportamientos afines y separando los de entropía distinta. Esto produce flujos de datos más homogéneos que, al ser alimentados a un compresor genérico, permiten una codificación más eficiente.

Importante: TDT no altera ningún bit de la información (es totalmente reversible), solo cambia su disposición para exponer patrones que antes estaban enmascarados. En el contexto de vectores de estado cuántico —que suelen ser casos límite de alta aleatoriedad— TDT buscaría, por ejemplo, si ciertos bytes de mantisa comparten distribuciones o si el byte de exponente tiene menor entropía debido a normalizaciones. Incluso si los datos son mayormente aleatorios, esta técnica puede detectar *no uniformidades* sutiles entre componentes y agruparlas para facilitar su compresión. Resulta particularmente atractiva como capa previa porque ofrece un equilibrio: en *datasets* de baja entropía conserva beneficios de modelos flotantes (predicciones locales), y en *datasets* de alta entropía se comporta como un reordenamiento que potencia a los compresores genéricos, habilitando ganancias modestas de compresión y mejoras de rendimiento en los *throughputs* de compresión y descompresión.

4.5 Otros Métodos de Optimización de Memoria

Además de la compresión sin pérdida, se han desarrollado enfoques alternativos para reducir el consumo de memoria en simuladores cuánticos, incluyendo técnicas basadas en redes tensoriales, poda de estados y optimización HPC.

Uno de los primeros enfoques en esta dirección fue propuesto por Markov y Shi (2008), quienes introdujeron el uso de redes tensoriales para simular circuitos cuánticos mediante contracción optimizada de tensores. En este método, el circuito se representa como un grafo tensorial, y su simulación se optimiza contrayendo secuencias de tensores en un orden eficiente. Esto permite representar ciertos circuitos con memoria subexponencial, aunque la efectividad del método depende de la estructura del entrelazamiento.

Posteriormente, Boixo, Isakov, Smelyanskiy, y Neven (2017) propusieron un modelo gráfico para circuitos cuánticos de baja profundidad basado en la eliminación de variables en grafos probabilísticos. Su técnica permitió simular circuitos aleatorios cercanos al régimen de supremacía cuántica en hardware clásico, extendiendo el tamaño de circuitos simulables sin necesidad de almacenar el estado cuántico completo. Sin embargo, este método pierde

eficacia conforme aumenta la profundidad del circuito.

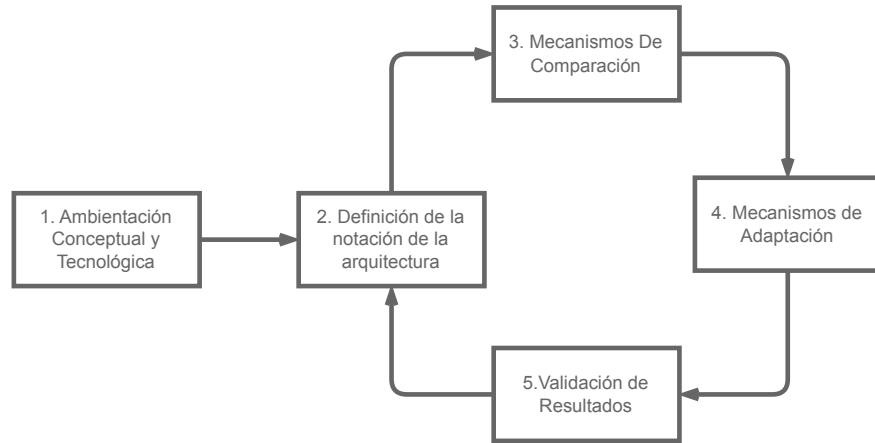
En paralelo, Pednault y cols. (2020) introdujeron técnicas de diferimiento de contracciones tensoriales en simulaciones HPC. Su propuesta permite manejar circuitos más grandes al intercambiar memoria por cómputo adicional, almacenando resultados intermedios en memoria secundaria y evitando la expansión total del estado cuántico en RAM. Esta estrategia ha sido clave en la simulación de circuitos con estructuras altamente no triviales, aunque su implementación requiere acceso a infraestructuras de supercomputación.

Finalmente, como se analizó previamente, la tesis doctoral de Díaz Toro (2025) introduce técnicas de compresión con pérdida de precisión en esquemas distribuidos de simulación cuántica. Si bien su propuesta constituye un hito en la reducción del consumo de memoria mediante bibliotecas como ZFP y SZ, el presente trabajo se orienta en una dirección distinta: se retoman fundamentos como el uso de vectores de estado y paralelismo distribuido con *MPI*, pero se propone como contribución original la exploración de técnicas de compresión **sin pérdida de precisión**. Esta línea de trabajo, no abordada en dicha tesis, busca preservar la fidelidad numérica total durante la simulación, evaluando el impacto empírico de algoritmos de compresión científica sin pérdida en el contexto de computación cuántica.

5 Metodología de la Investigación

Para este estudio, se ha seleccionado una metodología basada en investigación aplicada y evaluación experimental debido a la necesidad de diseñar, implementar y validar una estrategia de compresión sin pérdida de precisión en simuladores cuánticos. Dado que el problema a abordar implica la optimización del uso de memoria en simulaciones de alto consumo computacional, es fundamental seguir un enfoque sistemático que permita comparar la efectividad de la propuesta con otras estrategias existentes. Además, la metodología elegida permite realizar pruebas controladas sobre diferentes simuladores cuánticos y evaluar el impacto de la compresión en términos de memoria, rendimiento y exactitud numérica, garantizando así la reproducibilidad y validez de los resultados obtenidos.

Figura 1:
Metodología de investigación



5.1 Diseño

Comprende el estudio del estado del arte, la definición de requisitos y supuestos, la selección del simulador, y la planificación experimental y de integración técnica.

- **Revisión de literatura y estado del arte.** Levantamiento sistemático de técnicas de compresión sin pérdida de precisión para arreglos de punto flotante y su aplicabilidad en simulación cuántica de *vectores de estado*. Se identifican ventajas, limitaciones y brechas; se sintetizan los fundamentos teóricos y se establecen hipótesis de trabajo.
- **Criterios y procedimiento de selección del simulador cuántico.** Evaluación de alternativas (p. ej., QuEST, Intel-QS, qsim, Quantum++, TMFQS) con base en: (i) *apertura y modificabilidad* del código, (ii) *facilidad de integración* de estructuras/arrays personalizados para el vector de estado, (iii) *soporte a optimizaciones de memoria* y paralelismo (OpenMP/MPI/GPU), y (iv) *capacidad de instrumentación* (medición de memoria y tiempo). Se aplica una matriz de decisión ponderada y se documenta la trazabilidad de la elección.
- **Arquitectura de la solución y diseño de la estrategia de compresión.** Especificación de las estructuras de datos y puntos de inserción en el simulador (capa de

almacenamiento del vector de amplitudes y rutas críticas de acceso). Selección del/los algoritmo(s) de compresión sin pérdida de precisión o equivalentes numéricos que no alteren las amplitudes de referencia, así como políticas de (des)compresión y su impacto esperado en memoria, tiempo y paralelismo.

5.2 Implementación y optimización del modelo

La estrategia de compresión seleccionada se implementará en el simulador cuántico elegido, modificando su sistema de almacenamiento y gestión de memoria. Se realizarán optimizaciones para garantizar un desempeño eficiente, evitando costos computacionales excesivos debido a la aplicación de la compresión. Durante esta etapa, se llevarán a cabo las siguientes actividades:

- Modificación del código fuente del simulador para integrar la compresión sin pérdida.
- Optimización del sistema de almacenamiento para reducir el uso de memoria sin afectar el rendimiento.
- Validación inicial de la implementación mediante pruebas con circuitos de prueba.

5.3 Evaluación experimental

Se diseñará un conjunto de experimentos para evaluar el impacto de la estrategia de compresión en diferentes circuitos cuánticos. Los experimentos permitirán determinar si se logra un balance adecuado entre eficiencia computacional y exactitud numérica. En esta fase se realizarán las siguientes actividades:

- Definición de un conjunto de circuitos de prueba para evaluar la estrategia de compresión:
 - Transformada de Fourier Cuántica (QFT)
 - Algoritmo de búsqueda de Grover

- Ejecución de simulaciones con y sin compresión para medir su impacto en el rendimiento y el uso de memoria:

- Tiempo de ejecución total (segundos)
- Uso máximo de memoria (MB)
- Verificación de igualdad exacta del estado respecto a la referencia sin compresión:

$$\alpha_i^{(\text{ref})} = \alpha_i^{(\text{comp})} \quad \forall i.$$

- Error absoluto máximo como medida auxiliar de discrepancia:

$$e_{\text{abs}} = \max_i \left| \alpha_i^{(\text{ref})} - \alpha_i^{(\text{comp})} \right|.$$

- Análisis de los resultados obtenidos y comparación con enfoques tradicionales.

6 Simuladores cuánticos

6.1 Motivación y necesidad de un simulador base

El núcleo experimental de este trabajo consiste en diseñar e integrar técnicas de compresión *sin pérdida de precisión* sobre la estructura de datos más costosa en simulación cuántica tipo Schrödinger: el vector de estado (amplitudes complejas). Por lo tanto, el simulador seleccionado no es un detalle de implementación, sino una decisión metodológica que condiciona:

- la **viabilidad técnica** de insertar estructuras de almacenamiento comprimidas sin alterar la fidelidad numérica;
- el **costo de ingeniería** requerido para instrumentar mediciones reproducibles de memoria y tiempo;

- y la **capacidad de aislar variables** (compresión vs. paralelismo vs. optimizaciones internas) durante la evaluación experimental.

De hecho, dentro de la fase *Diseño* planteada en la metodología, se establece explícitamente que la selección debe basarse en: (i) apertura y modificabilidad del código, (ii) facilidad de integrar arreglos/estructuras personalizadas para el vector de estado, (iii) soporte a optimizaciones de memoria y paralelismo, y (iv) capacidad de instrumentación (medición de memoria y tiempo).

Bajo ese marco, se estudian cinco alternativas representativas por uso y disponibilidad en la comunidad: QuEST, Intel-QS, qsim, Quantum++ y TMFQSfullstate. Para asegurar trazabilidad, la decisión se formaliza con criterios y una matriz de decisión ponderada, de modo que la elección final sea justificable y reproducible.

6.2 Criterios de selección

Con el fin de elegir un simulador adecuado para integrar y evaluar compresión *sin pérdida de precisión* sobre el vector de estado, se definieron los criterios de la tabla 1. Estos criterios operacionalizan los lineamientos metodológicos del diseño (modificabilidad, integración de estructuras, optimización/paralelismo e instrumentación), y además incorporan requisitos prácticos de ingeniería para reducir riesgo técnico durante la implementación.

Tabla 1:

Criterios usados para la selección del simulador cuántico

	Criterio	Explicación	Peso
C1	Vector de estado apto para compresión sin pérdida	El simulador debe representar el vector de estado como arreglos contiguos de valores de punto flotante en doble precisión (amplitudes complejas), de manera que la compresión sin pérdida se pueda aplicar directamente sobre dicha estructura sin cambios semánticos.	0.10
C2	Facilidad de integración de estructuras/arrays personalizados	El código debe permitir reemplazar o envolver el almacenamiento del vector de amplitudes (p. ej. insertar un contenedor comprimido) con impacto controlado en el resto del simulador. Este criterio prioriza código modular y rutas críticas identificables.	0.25
C3	Soporte y afinidad con optimización de memoria	Se valora que el simulador esté concebido para experimentar con estrategias de memoria (p. ej. compresión, poda/pruning, particionamiento o variantes del almacenamiento), reduciendo esfuerzo para insertar y validar la estrategia propuesta.	0.20
C4	Complejidad de construcción y dependencias	Un proceso de compilación simple y pocas dependencias facilitan reproducibilidad y portabilidad de experimentos. Se penalizan toolchains complejas cuando incrementan el esfuerzo de integración e instrumentación.	0.10
C5	Capacidad de instrumentación (tiempo/memoria)	El simulador debe permitir medir de forma fina (por secciones) el consumo de memoria y el tiempo de ejecución, idealmente en puntos cercanos al acceso al vector de estado.	0.15
C6	Cobertura funcional mínima para benchmarks	Debe ser posible ejecutar circuitos estándar de evaluación (p. ej. QFT y/o Grover) con un conjunto mínimo de compuertas, para poder comparar desempeño con y sin compresión.	0.10
C7	Paralelismo relevante (CPU / OpenMP / MPI)	Se considera valioso poder evaluar la interacción compresión-paralelismo. Se prioriza soporte a OpenMP/MPI cuando el objetivo incluye escalamiento o comparación bajo concurrencia.	0.10

Los pesos fueron definidos para privilegiar la **integración del almacenamiento del vector de estado** (C2) y la **afinidad con experimentación en memoria** (C3), dado

que son los factores que más reducen el riesgo técnico de insertar compresión sin pérdida sin distorsionar el comportamiento del simulador.

6.3 Alternativas de simuladores evaluadas

A continuación, se resumen las herramientas evaluadas, enfatizando sus objetivos y características relevantes para este trabajo.

- **QuEST (Quantum Exact Simulation Toolkit):** Es un simulador de alto rendimiento para vectores de estado y matrices de densidad, con soporte híbrido de paralelismo que puede incluir multihilo, GPU y ejecución distribuida. Se caracteriza por estar orientado a rendimiento en múltiples arquitecturas de cómputo. (*The Quantum Exact Simulation Toolkit (QuEST) Documentation*, s.f.; *QuEST: Quantum Exact Simulation Toolkit*, s.f.)
- **Intel-QS (Intel Quantum Simulator):** Simulador de circuitos optimizado para aprovechar arquitecturas multinúcleo y multinodo; usa representación completa del estado, y está diseñado para escalar mediante paralelismo y distribución. (*Intel Quantum Simulator (Intel-QS) Documentation*, s.f.)
- **qsim (Google):** Librería de C++/Python para simulación *full state-vector*, ampliamente usada en integración con Cirq. En su documentación se describe explícitamente como simulador de vector de estado completo, donde el costo crece proporcionalmente al número de amplitudes 2^n . (*qsim: Fast C++ and Python library for state-vector simulation of quantum circuits*, s.f.; *qsim Cirq Interface Documentation*, s.f.)
- **Quantum++:** Librería moderna en C++ (enfocada en facilidad de uso y portabilidad) para simulación de procesos cuánticos; se distribuye como librería *header-only* y ofrece ejecución multihilo, siendo útil para prototipado. (Gheorghiu, 2018)
- **TMFQSfullstate:** Prototipo de simulador implementado en C++ con enfoque explícito en estudiar estrategias de consumo de memoria (representación full-state, poda de

estados, paralelismo compartido y distribuido, y compresión). En particular, el trabajo metodológico que acompaña este simulador reporta escenarios con OpenMP, MPI y la incorporación de compresión para evaluar huella de memoria. (G. J. Díaz, 2024; G. Díaz, 2025; Díaz Toro, 2024)

En conjunto, las alternativas cubren desde herramientas HPC maduras (QuEST, Intel-QS, qsim) hasta librerías generales (Quantum++) y un prototipo deliberadamente diseñado para experimentar con estrategias de memoria (TMFQSfullstate).

6.4 Matriz de evaluación ponderada

Se aplicó una matriz de decisión ponderada para evaluar cada alternativa según los criterios de la tabla 1. La calificación se asignó en una escala discreta de 1 a 5, donde 5 indica cumplimiento excelente del criterio y 1 un cumplimiento deficiente para los objetivos de integración e instrumentación de compresión sin pérdida.

Tabla 2:

Evaluación de alternativas (escala 1–5) y total ponderado

Criterio	Peso	TMFQS	QuEST	Intel-QS	qsim	Quantum++
C1	0.10	5	4	4	4	4
C2	0.25	5	3	2	2	3
C3	0.20	5	3	3	2	2
C4	0.10	4	3	2	2	5
C5	0.15	5	3	2	2	3
C6	0.10	3	5	5	5	4
C7	0.10	4	5	5	4	3
Total	1.00	4.60	3.50	3.00	2.70	3.10

Los resultados muestran que, aunque QuEST, Intel-QS y qsim ofrecen excelente cobertura funcional y paralelismo, la elección para este trabajo debe priorizar el costo de modificación del almacenamiento del vector de estado y la instrumentación cercana a rutas críticas. Bajo

ese objetivo, TMFQSfullstate obtiene el mayor puntaje global, principalmente por C2, C3 y C5.

6.5 Selección del simulador: TMFQSfullstate

Partiendo de la matriz de la tabla 2, se selecciona **TMFQSfullstate** como simulador base para este trabajo.

La decisión se justifica por cuatro razones principales:

- **Diseño orientado a experimentación de memoria (C3):** TMFQSfullstate se presenta como un prototipo cuyo foco explícito es evaluar consumo de memoria bajo distintas estrategias (full-state, OpenMP, MPI y escenarios con compresión), lo cual se alinea directamente con el objetivo de insertar compresión sin pérdida sobre el vector de estado. (G. J. Díaz, 2024; G. Díaz, 2025)
- **Alta modificabilidad del núcleo (C2) e instrumentación (C5):** al ser un prototipo compacto y de propósito investigativo, reduce el esfuerzo necesario para ubicar puntos de inserción (capa de almacenamiento del vector de amplitudes y rutas críticas) y para instrumentar mediciones finas de tiempo/memoria, tal como se exige en la fase de diseño metodológico.
- **Compatibilidad con arreglos de doble precisión (C1):** el material metodológico asociado al prototipo describe implementaciones basadas en arreglos en doble precisión para compuertas y estructuras numéricas, lo que facilita plantear compresión sin pérdida sobre datos de punto flotante sin introducir conversiones o cuantizaciones. (G. J. Díaz, 2024; G. Díaz, 2025)
- **Escenarios comparables de paralelismo (C7):** la disponibilidad de variantes con OpenMP/MPI permite evaluar la interacción entre compresión y paralelismo, y contrastar contra simuladores HPC sin que el paralelismo sea un factor ausente en el entorno experimental. (G. J. Díaz, 2024)

En consecuencia, TMFQSfullstate ofrece el mejor balance entre *control experimental* y *esfuerzo de integración*, minimizando riesgo técnico para implementar y evaluar compresión sin pérdida en el vector de estado, manteniendo al mismo tiempo una base suficientemente realista para comparar memoria y rendimiento.

7 Selección de una biblioteca de compresión sin pérdida para integración en TMFQSfullstate

7.1 Motivación y contexto de selección

Un simulador tipo Schrödinger puede particionar el vector de estado en bloques independientes, mantener una caché de bloques descomprimidos y aplicar una política de descompresión bajo demanda para evitar reconstruir el estado completo en cada operación. Sin embargo, para materializar esa arquitectura en un simulador real es necesario seleccionar una biblioteca de compresión concreta que se adapte bien al acceso parcial por bloques y al patrón repetido de lectura, modificación y escritura de amplitudes.

Por esta razón, la decisión relevante es qué biblioteca de compresión sin pérdida facilita una integración eficiente en una ruta crítica de simulación cuántica. En este contexto, interesan especialmente las soluciones que trabajen con bloques independientes, permitan incorporar filtros previos sobre datos de punto flotante y minimicen la latencia de descompresión cuando solo una fracción del vector de estado debe pasar temporalmente a representación densa.

7.2 Criterios de selección

La selección de bibliotecas de compresión sin pérdida se realizó mediante los criterios de la Tabla 3. Estos criterios recogen propiedades relevantes para la compresión de vectores de estado representados como arreglos numéricos de gran tamaño.

Como condiciones mínimas de inclusión se consideraron la operación sin pérdida, la disponibilidad de interfaces utilizables desde C/C++, un licenciamiento compatible con el uso

académico y la posibilidad de comprimir datos residentes en memoria. Las bibliotecas comparadas satisfacen estas condiciones.

Tabla 3:

Criterios para la selección de bibliotecas de compresión sin pérdida

	Criterio	Explicación	Peso
C1	Efectividad de compresión sobre arreglos numéricos	Se evalúa la capacidad de la biblioteca para reducir el tamaño de arreglos homogéneos de punto flotante, condición central en un simulador basado en vectores de estado.	0.25
C2	Velocidad de descompresión	La recuperación rápida de datos es importante cuando los valores comprimidos deben reutilizarse durante la simulación. Este criterio mide qué tan favorable es la biblioteca para escenarios sensibles a la latencia de lectura.	0.20
C3	Velocidad de compresión	Considera el costo temporal de comprimir los datos, ya que una estrategia útil para el ahorro de memoria no debe introducir una penalización excesiva en el tiempo total de simulación.	0.15
C4	Facilidad de integración e instrumentación en C/C++	La biblioteca debe ofrecer una API estable, reutilizable desde el simulador y con suficiente control sobre parámetros operativos, buffers auxiliares y niveles de configuración, reduciendo el riesgo de integración.	0.15
C5	Madurez y adopción	Se valora la estabilidad del ecosistema de la biblioteca, la disponibilidad de documentación, la adopción en entornos reales y la probabilidad de mantenimiento a mediano plazo.	0.10
C6	Adecuación a datos científicos en punto flotante	Para arreglos numéricos homogéneos, se valora la existencia o integración razonable de mecanismos que mejoren el aprovechamiento de regularidades binarias sin comprometer el carácter sin pérdida de la compresión.	0.15

Los pesos priorizan propiedades centrales del problema: ahorro efectivo de memoria sobre datos numéricos, bajo costo de acceso, integración práctica y adecuación al contexto científico.

7.3 Alternativas de compresión sin pérdida consideradas

Las alternativas consideradas corresponden a bibliotecas o familias de bibliotecas utilizables desde C/C++ y con potencial de integrarse en una arquitectura de almacenamiento comprimido por bloques.

- **Blosc2:** Biblioteca orientada al manejo de bloques comprimidos en memoria, con soporte para filtros como *shuffle* y *bitshuffle*, y posibilidad de usar distintos *codecs* en cada bloque. Estas características la ubican dentro del grupo de herramientas diseñadas para datos numéricos y estructuras particionadas.
- **Zstandard (Zstd):** Biblioteca generalista ampliamente utilizada en compresión sin pérdida. Se caracteriza por ofrecer distintas configuraciones de velocidad y ratio, y puede integrarse como compresor dentro de flujos de trabajo más amplios definidos por la aplicación.
- **LZ4:** Biblioteca orientada a compresión y descompresión de baja latencia. Su uso es frecuente en contextos donde la rapidez del acceso tiene un peso importante, y al igual que otras bibliotecas generalistas puede emplearse como componente dentro de esquemas de almacenamiento definidos externamente.
- **zlib/Deflate:** Solución madura y ampliamente disponible, con amplia adopción en múltiples entornos de software. Representa una referencia clásica dentro de la compresión sin pérdida y permite contrastar alternativas más recientes con una base tecnológica bien establecida.

Aunque otras alternativas también podrían adaptarse al problema, estas cuatro cubren el espectro relevante para este trabajo.

7.4 Matriz de evaluación ponderada

La evaluación se presenta mediante una matriz ponderada basada en los criterios de la Tabla 3. La escala de 1 a 5 expresa el grado de adecuación de cada biblioteca frente a compresión sin pérdida de arreglos numéricos, costo temporal de acceso e integración práctica en software científico escrito en C/C++.

Tabla 4:
Evaluación de bibliotecas de compresión sin pérdida (escala 1–5) y total ponderado

Criterio	Peso	Blosc2	Zstd	LZ4	zlib
C1 (Efectividad en datos numéricos)	0.25	4	4	2	3
C2 (Descompresión)	0.20	5	4	5	2
C3 (Compresión)	0.15	4	4	5	2
C4 (Integración C/C++)	0.15	5	5	5	5
C5 (Madurez y adopción)	0.10	4	5	5	5
C6 (Optimización para datos numéricos)	0.15	5	2	1	1
Total	1.00	4.50	3.95	3.65	2.85

Los resultados de la matriz muestran perfiles diferenciados entre las bibliotecas evaluadas. Zstd y LZ4 presentan un buen comportamiento general como compresores de alta velocidad, zlib conserva interés como referencia madura y ampliamente adoptada, y Blosc2 obtiene la mayor puntuación total por su buen balance entre costo de acceso, integración y adecuación a arreglos numéricos homogéneos.

7.5 Selección del algoritmo recomendado

Con base en la Tabla 4, se selecciona **Blosc2** como biblioteca principal para la implementación del mecanismo de compresión sin pérdida.

Zstd y LZ4 presentan ventajas claras como compresores generalistas de alta velocidad, mientras que zlib ofrece una base madura y ampliamente difundida. Dentro del conjunto de

criterios definidos para este trabajo, Blosc2 obtiene el mejor balance entre descompresión rápida, integración en C/C++ y adecuación al tratamiento de datos numéricos.

En consecuencia, la implementación concreta desarrollada en este trabajo adopta Blosc2 como realización de una estrategia general de compresión en memoria para vectores de estado, basada en particionamiento, acceso localizado y actualización selectiva de los datos comprimidos.

8 Patrones en vectores de estado de algoritmos cuánticos relevantes para compresión

8.1 Introducción

La simulación cuántica de estado completo representa el sistema mediante un vector de amplitudes complejas cuyo tamaño crece como 2^n (siendo n el número de qubits). En este contexto, la *compresibilidad* del vector depende fuertemente de los patrones numéricos que generan los algoritmos: simetrías, valores repetidos, regiones con ceros exactos o distribuciones altamente estructuradas tienden a favorecer la compresión; por el contrario, estados densos con amplitudes variadas y poco redundantes tienden a reducir la efectividad de compresores sin pérdida.

En las subsecciones siguientes se describen patrones típicos de varios algoritmos conocidos y su impacto esperado en compresión de vectores de estado.

8.2 Búsqueda de Grover: amplitudes uniformes y estados de baja entropía

Grover inicia preparando una superposición uniforme sobre $N = 2^n$ estados base, usualmente aplicando compuertas de Hadamard a cada qubit. En esa preparación ideal, todas las amplitudes tienen el mismo valor (magnitud constante), por lo que el vector de estado queda compuesto por un único valor repetido N veces, un caso altamente favorable para compresión sin pérdida (PennyLane, 2023).

Tras cada iteración (oráculo + difusión), la estructura permanece altamente regular:

cuando existe un único elemento marcado, el vector puede describirse con dos categorías principales de amplitud (una asociada al estado marcado y otra común al resto de estados no marcados). Esta simetría reduce el número de valores distintos en el vector, lo cual incrementa la redundancia explotable por compresión.

De manera empírica, se ha reportado que la compresión permite escalar simulaciones de Grover significativamente más allá de lo que permitiría almacenar el vector sin comprimir. Por ejemplo, Wu y cols. muestran una reducción drástica del requerimiento de memoria para una simulación de Grover de 61 qubits al combinar técnicas de compresión con ejecución en supercomputación (Wu y cols., 2019b).

8.3 Transformada Cuántica de Fourier: patrones de fase y baja redundancia

La Transformada Cuántica de Fourier (QFT) aplicada a un estado base $|k\rangle$ produce una superposición con magnitudes uniformes pero fases dependientes del índice. En su forma ideal, la amplitud de cada componente $|x\rangle$ puede expresarse como:

$$a_x = \frac{1}{\sqrt{N}} e^{\frac{2\pi i k x}{N}},$$

lo cual implica que, aunque la magnitud sea constante, las partes real e imaginaria varían de forma oscilatoria con x .

Desde la perspectiva de almacenamiento, este comportamiento tiende a generar arreglos *densos* donde la mayoría de entradas son no nulas y muchas de ellas difieren entre sí. En consecuencia, la redundancia directa (por repetición exacta) suele ser baja para compresión estrictamente sin pérdida. En un estudio de integración de compresión en simulación cuántica se observa que, a medida que avanza un circuito QFT, el ratio de compresión puede caer desde valores altos en etapas iniciales hasta valores cercanos a 1 (poca o nula compresión efectiva) en etapas finales (Wu, Di, y cols., 2018). Este comportamiento ilustra que los estados generados por QFT pueden ser un caso exigente para compresión sin pérdida, especialmente

cuando el circuito densifica el vector de estado.

8.4 Algoritmo de Shor: periodicidad, dispersión y picos

Shor combina (i) una etapa de exponenciación modular que induce periodicidad y (ii) una QFT que transforma esa periodicidad en una distribución con picos relacionados con el período. En términos cualitativos, antes de aplicar QFT suelen aparecer patrones donde solo ciertos índices compatibles con una estructura periódica presentan amplitud significativa; dependiendo de la instancia, esto puede producir regiones con ceros exactos o repeticiones regulares que favorecen la compresión.

Tras QFT, la distribución tiende a concentrarse alrededor de múltiplos específicos asociados a la frecuencia/período, generando picos dominantes y muchos valores de magnitud pequeña. En compresión estrictamente sin pérdida, la ganancia depende de si existen ceros exactos o repeticiones; cuando predominan muchos valores pequeños pero distintos, la redundancia disminuye. Estos escenarios se alinean con el fenómeno observado en circuitos que incrementan progresivamente la densidad del estado: conforme aparecen más amplitudes no nulas y variadas, el ratio de compresión disminuye (Wu, Di, y cols., 2018).

8.5 Deutsch–Jozsa y Simon: superposiciones estructuradas y esparsidad

Varios algoritmos tempranos construyen superposiciones altamente estructuradas mediante oráculos e interferencia.

En Deutsch–Jozsa, la computación induce cancelaciones que llevan el estado hacia un resultado fuertemente concentrado en un subconjunto pequeño de estados base (en casos ideales, un estado base dominante). Este tipo de concentración implica esparsidad (muchos ceros exactos), lo que es altamente favorable para compresión sin pérdida.

En Simon, tras medir el registro de salida, el registro de entrada colapsa a una superposición de dos estados relacionados por una máscara secreta. Ese estado intermedio es extremadamente esparso (dos entradas no nulas) y, por tanto, muy compresible. Posteriormente,

la aplicación de Hadamards produce una superposición uniforme sobre un subconjunto que satisface una condición lineal; nuevamente aparecen patrones de ceros exactos y amplitudes repetidas. Una exposición divulgativa describe este comportamiento estructurado y la naturaleza de las superposiciones restringidas en Simon (Gupta, 2025).

8.6 Circuitos variacionales y circuitos aleatorios: entre regularidad y alta entropía

En aplicaciones NISQ, algunos circuitos variacionales producen distribuciones de amplitud sesgadas por la estructura del problema, lo cual puede inducir regularidades (por ejemplo, subconjuntos dominantes o restricciones que eliminan estados). Esto puede mejorar la compresibilidad frente a escenarios plenamente aleatorios.

Por el contrario, circuitos aleatorios profundos tienden a generar estados altamente entrelazados y con amplitudes densas, donde la diversidad de valores reduce la redundancia. En simulación con compresión, este fenómeno se manifiesta como una caída del ratio a medida que el circuito genera más amplitudes no nulas y variadas, tal como se evidencia en evaluaciones de compresión sobre circuitos que densifican el vector de estado (por ejemplo, QFT) (Wu, Di, y cols., 2018).

8.7 Implicaciones para compresión sin pérdida en simulación de estado completo

Los ejemplos anteriores sugieren una heurística práctica: algoritmos que preservan simetrías fuertes (valores repetidos), esparsidad (ceros exactos) o distribuciones con pocas categorías de amplitud tienden a ser más favorables para compresión sin pérdida. En cambio, transformaciones que producen estados densos y con amplitudes altamente variadas limitan la efectividad de compresores sin pérdida.

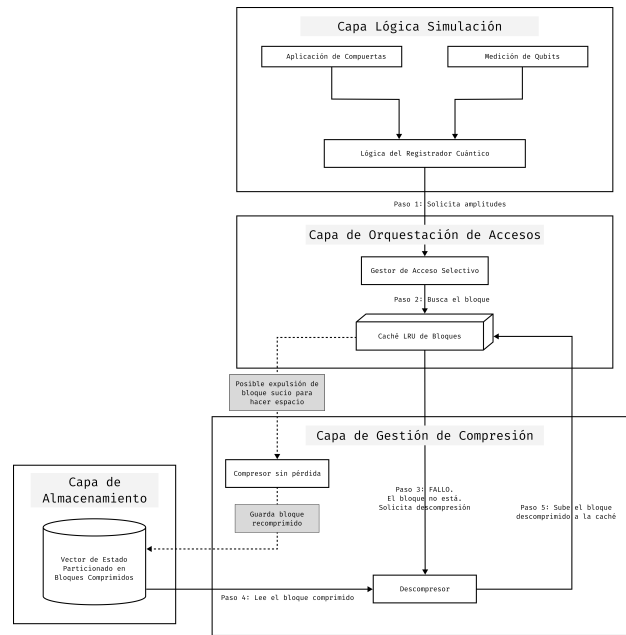
Esta relación patrón-compresibilidad es un insumo útil para diseñar experimentos y seleccionar benchmarks representativos al evaluar técnicas de reducción de memoria en si-

muladores cuánticos. Resultados experimentales sobre compresión en simulación de circuitos muestran precisamente que, en presencia de estados densos y sin reglas simples de distribución, el ratio puede acercarse a 1 (Wu, Di, y cols., 2018), mientras que en algoritmos con alta regularidad (como Grover) pueden observarse reducciones de memoria sustanciales (Wu y cols., 2019b).

9 Diseño de simulación para compresión sin pérdida en simuladores tipo Schrödinger

Una vez seleccionado el simulador base y definida la biblioteca de compresión a emplear, el siguiente paso consiste en establecer una arquitectura de simulación que permita introducir compresión sin pérdida sin alterar la semántica del simulador de estado completo. En esta sección se presenta una propuesta de diseño en niveles de abstracción, pensada para ser reutilizable en cualquier simulador tipo Schrödinger cuyo estado cuántico se represente como un vector de amplitudes complejas en doble precisión.

Figura 2: Arquitectura abstracta para integración de compresión sin pérdida en simuladores tipo Schrödinger



9.1 Objetivos del diseño

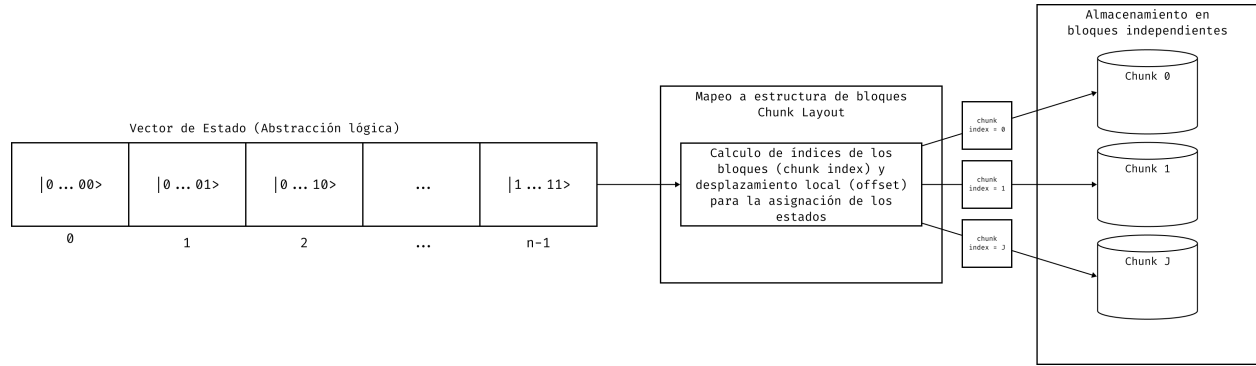
El diseño propuesto busca satisfacer cuatro objetivos simultáneos. En primer lugar, reducir la huella de memoria del vector de estado, que crece exponencialmente con el número de qubits. En segundo lugar, preservar la exactitud numérica del simulador, evitando pérdidas de precisión en las amplitudes complejas. En tercer lugar, impedir que cada compuerta obligue a reconstruir el vector de estado completo en memoria densa. Finalmente, se requiere que la sobrecarga temporal introducida por la compresión permanezca controlada y pueda ser ajustada mediante parámetros de diseño.

Estos objetivos conducen a una separación clara entre la lógica del simulador y la estrategia de almacenamiento. La lógica cuántica continúa operando sobre amplitudes complejas, compuertas y mediciones, mientras que la representación física del vector puede variar entre una forma densa y una forma comprimida por bloques, siempre que ambas expongan el mismo comportamiento observable.

9.2 Representación por bloques del vector de estado

La decisión estructural principal consiste en particionar el vector de estado en bloques independientes de tamaño fijo. Cada bloque agrupa un número determinado de estados base consecutivos y almacena sus partes real e imaginaria como un arreglo contiguo de números en punto flotante. Esta organización convierte el problema global de compresión del estado completo en una colección de problemas locales sobre subarreglos más pequeños.

La ventaja de esta representación es doble. Por una parte, cada bloque puede comprimirse y descomprimirse sin depender del contenido de los demás, lo que facilita actualizaciones localizadas. Por otra, el tamaño del bloque se convierte en un parámetro de diseño que permite balancear dos efectos opuestos: bloques más grandes tienden a favorecer el ratio de compresión, mientras que bloques más pequeños reducen el costo de acceso cuando solo una fracción del vector es requerida.

Figura 3: Particionamiento abstracto del vector de estado en bloques independientes

9.3 Descompresión selectiva y estrategia de acceso

En una arquitectura densa tradicional, cualquier compuerta opera directamente sobre el arreglo completo de amplitudes. En contraste, cuando el estado está comprimido por bloques, la política de acceso debe asegurar que solo se materialicen en memoria densa los bloques realmente tocados por la operación actual. De este modo, una compuerta de uno o dos qubits no desencadena una descompresión global, sino un proceso local de adquisición de bloques, modificación y posterior recompresión.

Esta idea conduce a una estrategia de descompresión selectiva. Cada acceso a una amplitud o a un conjunto de amplitudes se traduce en la identificación de los bloques involucrados. Si un bloque ya se encuentra disponible en forma descomprimida, la operación se realiza directamente sobre ese buffer. Si el bloque aún está almacenado en forma comprimida, primero se descomprime, se ejecuta la actualización necesaria y luego se conserva temporalmente en memoria de trabajo para evitar recomprimirlo de inmediato si es probable que vuelva a usarse en operaciones posteriores.

Desde el punto de vista algorítmico, esta estrategia es especialmente relevante para compuertas que inducen accesos repetidos a subconjuntos del estado, así como para secuencias cortas de compuertas sobre qubits cercanos. En esos casos, mantener un subconjunto pequeño del vector en memoria descomprimida puede amortiguar el costo de las transiciones entre

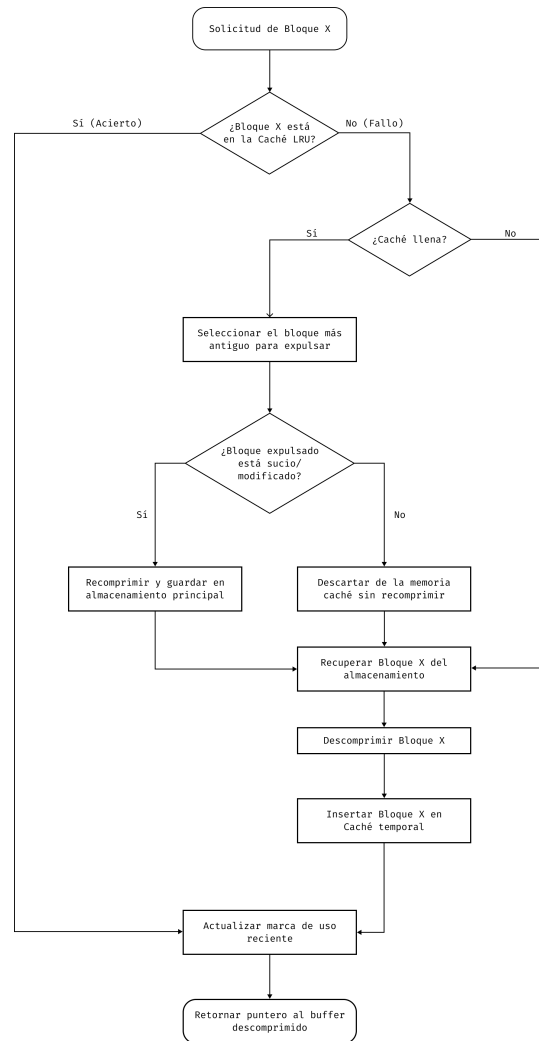
representación comprimida y representación densa.

9.4 Caché LRU de bloques descomprimidos

Para sostener la estrategia anterior se introduce una caché de bloques descomprimidos. Su función es actuar como un conjunto de trabajo temporal que almacena los bloques accedidos recientemente. Cuando una operación necesita un bloque, primero consulta la caché. Si el bloque está presente, ocurre un acierto y se evita una nueva descompresión. Si el bloque no está presente, ocurre un fallo, el bloque se recupera desde almacenamiento comprimido y se incorpora al conjunto de trabajo.

La política de reemplazo adoptada es del tipo *Least Recently Used* (LRU). Bajo esta política, cuando la caché está llena y se necesita espacio para un nuevo bloque, se expulsa el bloque cuyo último acceso sea el más antiguo. La elección de LRU es adecuada para este contexto porque muchas secuencias de compuertas presentan localidad temporal: un bloque recién usado tiene mayor probabilidad de volver a ser requerido en el corto plazo que uno accedido hace muchas operaciones.

Si el bloque expulsado fue modificado, debe recomprimirse antes de abandonar la caché. Si no fue modificado, puede descartarse sin costo de escritura. Esta distinción entre bloques limpios y sucios evita recompresiones innecesarias y reduce la sobrecarga total del sistema.

Figura 4: Esquema conceptual de la caché LRU de bloques descomprimidos

9.5 Flujo de compresión y descompresión

La compresión propuesta se entiende como un pipeline de etapas reversibles sobre cada bloque. Primero, el bloque en doble precisión se organiza como un arreglo contiguo de valores. Luego puede aplicarse una transformación previa sobre la representación binaria de estos datos, por ejemplo una variante de *shuffle* o *bitshuffle*, con el objetivo de exponer regularidades entre bytes o planos de bits. Finalmente, el flujo transformado se entrega a un compresor sin pérdida de baja latencia.

El proceso inverso ocurre solo cuando el bloque es requerido por una operación. El bloque

comprimido se descomprime, se invierte la transformación previa y se obtiene nuevamente el arreglo denso de amplitudes listo para ser manipulado por la lógica del simulador. La clave del diseño es que esta secuencia solo se ejecuta sobre los bloques activos y no sobre la totalidad del vector.

Para operaciones que afectan amplitudes pertenecientes a un mismo bloque, la actualización es enteramente local. Cuando la operación cruza bloques, la arquitectura requiere adquirir simultáneamente los bloques involucrados, descomprimir ambos, ejecutar la actualización y decidir luego cuáles mantener temporalmente en caché y cuáles devolver al almacenamiento comprimido. Este es el punto donde el tamaño de la caché y la granularidad de los bloques influyen de manera más visible sobre el costo temporal.

Espacio reservado para una figura del flujo “adquirir bloque – descomprimir – modificar – marcar como sucio – recomprimir” y su variante para operaciones entre bloques distintos.

Figura 5: Flujo abstracto de acceso y actualización de bloques comprimidos

9.6 Ventajas, parámetros y limitaciones del diseño

El principal beneficio de esta arquitectura es que desacopla el ahorro de memoria de la necesidad de reconstruir por completo el vector de estado en cada operación. Además, introduce parámetros claros y controlables: tamaño del bloque, número de entradas en caché, política de reemplazo y tipo de compresor sin pérdida. Esto permite adaptar la arquitectura a diferentes patrones de acceso y a distintas disponibilidades de memoria.

No obstante, la arquitectura también presenta limitaciones. Si los estados cuánticos generados por el algoritmo son poco compresibles, la reducción de memoria puede resultar modesta. Del mismo modo, si las compuertas inducen accesos dispersos sobre muchos bloques distintos, la tasa de fallos de caché puede crecer y aumentar la cantidad de descompresiones. Por esta razón, el análisis experimental posterior debe estudiar el compromiso entre ratio de compresión, latencia y tamaño del conjunto de trabajo.

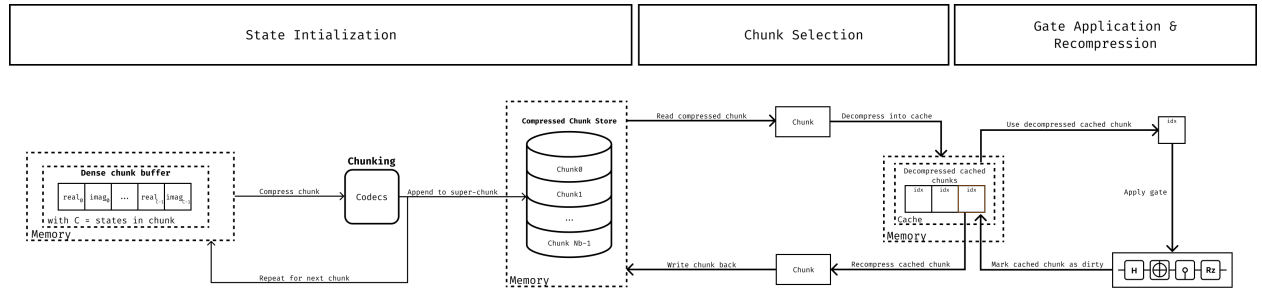
En conjunto, los elementos anteriores delimitan un espacio de compromiso entre ahorro

de memoria, granularidad de acceso y costo temporal de compresión y descompresión. Sobre esa base, la siguiente sección presenta la materialización de estas decisiones en un mecanismo de almacenamiento comprimido apoyado en Blosc.

10 Desarrollo e implementación del backend basado en Blosc

Esta sección presenta el desarrollo del mecanismo de almacenamiento comprimido basado en Blosc incorporado en TMFQSfullstate. La descripción se organiza alrededor de la disposición en bloques del vector de estado, la caché LRU de bloques descomprimidos y la estrategia de acceso local utilizada durante la simulación.

Figura 6: Arquitectura del desarrollo implementado para el backend basado en Blosc



10.1 Contexto de implementación

La integración se realizó sobre TMFQSfullstate, manteniendo inalterada la interfaz lógica del simulador y concentrando los cambios en la capa de almacenamiento del vector de estado. Esta decisión permite que las operaciones de más alto nivel sigan trabajando con amplitudes complejas, compuertas y mediciones, mientras que la representación física del estado puede cambiar de densa a comprimida sin afectar la semántica del simulador.

En ese contexto, el backend desarrollado se apoya en tres decisiones concretas. La primera es representar el estado mediante bloques de amplitudes. La segunda es emplear Blosc2 como biblioteca de compresión sin pérdida para cada bloque. La tercera es introducir una caché LRU de bloques descomprimidos para amortiguar accesos repetidos durante la aplicación de

compuertas.

10.2 Backend de almacenamiento comprimido basado en Blosc

La implementación realizada adopta Blosc2 como realización concreta de la estrategia abstracta descrita antes. El estado cuántico completo se divide en bloques de tamaño configurable, y cada bloque se almacena de forma comprimida. Esta organización permite que la descompresión ocurra sobre una unidad local y no sobre el vector completo.

Una ventaja importante de esta elección es que Blosc2 proporciona soporte directo para compresión rápida por bloques, así como filtros previos que mejoran la compresibilidad de arreglos numéricos. En el contexto del simulador, esto se traduce en una mejor adecuación al patrón “descomprimir solo lo necesario, modificar localmente y recomprimir cuando corresponda”.

Además, el backend expone parámetros de configuración relevantes para el experimento: número de estados por bloque, nivel de compresión, número de hilos y cantidad de entradas de caché. Estos parámetros permiten ajustar el compromiso entre uso de memoria y costo temporal, y por ello deben ser reportados explícitamente cuando se presenten resultados experimentales.

10.3 Disposición en bloques y mapeo del vector de estado

La primera parte del desarrollo consistió en introducir una disposición explícita del vector de estado en bloques contiguos. Cada bloque agrupa una cantidad fija de estados base y almacena sus componentes real e imaginaria en forma intercalada. Con ello, cualquier estado base puede mapearse a un índice de bloque y a un desplazamiento local dentro de ese bloque.

Esta capa de mapeo es necesaria para que el backend identifique rápidamente qué bloque debe adquirirse ante una operación de lectura, escritura o aplicación de compuerta. También permite tratar el último bloque de manera especial cuando el número total de estados no llena exactamente una unidad completa de almacenamiento.

Espacio reservado para una figura que muestre el mapeo entre índices globales del vector de estado, bloques físicos y desplazamientos locales dentro de cada bloque.

Figura 7: Mapeo entre estados base y bloques físicos en el backend comprimido

10.4 Caché LRU de bloques descomprimidos

El segundo componente clave del desarrollo fue la incorporación de una caché de bloques descomprimidos. Su objetivo es mantener en memoria de trabajo los bloques usados recientemente, de modo que múltiples operaciones sucesivas puedan reutilizarlos sin repetir el proceso de descompresión.

La política de reemplazo adoptada es LRU. Cada acceso actualiza la marca de uso reciente del bloque, y cuando la caché se llena se selecciona para expulsión el bloque con el acceso más antiguo. Si el bloque expulsado fue modificado, se marca como sucio y se recomprime antes de salir de la caché; si no hubo modificaciones, puede descartarse sin costo adicional de escritura.

Desde el punto de vista de la simulación, esta decisión es importante porque muchas secuencias de compuertas reutilizan un subconjunto reducido del estado. En tales casos, una caché pequeña pero bien administrada puede reducir de forma significativa la frecuencia de descompresiones y recompresiones.

10.5 Estrategia de acceso y actualización local

La implementación concreta sigue el siguiente esquema operativo. Cuando una operación necesita una amplitud o un par de amplitudes, primero determina los bloques implicados. Después consulta la caché. Si el bloque ya está disponible, reutiliza el buffer descomprimido. Si no está disponible, lo recupera desde almacenamiento comprimido, lo inserta en la caché y continúa con la operación.

Una vez modificado un bloque, este no se recomprime inmediatamente por defecto. En lugar de ello, se mantiene en la caché marcado como sucio hasta que sea necesario liberarlo o

hasta que la operación actual requiera un vaciado explícito. Esta política evita recompresiones prematuras cuando varias compuertas consecutivas afectan el mismo conjunto de bloques.

Para compuertas cuyas amplitudes involucradas residen en un mismo bloque, el procedimiento es completamente local. En cambio, para operaciones que afectan amplitudes distribuidas en bloques distintos, el backend debe adquirir ambos bloques y gestionar su permanencia en la caché. Este caso hace evidente la necesidad de al menos dos ranuras activas y explica por qué la caché no puede reducirse a una sola entrada.

Espacio reservado para una secuencia de pasos de acceso: localizar bloque, verificar caché, descomprimir, modificar, marcar como sucio y eventualmente recomprimir al expulsar.

Figura 8: Secuencia operativa del backend basado en Blosc

El mecanismo descrito en este capítulo establece la base operativa para medir el efecto de la compresión sobre circuitos representativos. La sección siguiente presenta la evaluación experimental realizada sobre Grover y QFT, así como la comparación cuantitativa entre la estrategia sin compresión, la estrategia sin pérdida basada en Blosc y una estrategia con pérdida controlada basada en ZFP.

11 Evaluación experimental y resultados

La evaluación experimental se orientó a cuantificar el efecto de la compresión sobre tres dimensiones: uso máximo de memoria, tiempo total de ejecución y error numérico respecto a la simulación de referencia sin compresión. El análisis se realizó sobre dos familias de circuitos con perfiles distintos de acceso al vector de estado: búsqueda de Grover y transformada cuántica de Fourier (QFT). En ambos casos se ejecutaron conjuntos de pruebas con distintos tamaños de problema y con diferentes entradas, manteniendo la misma configuración experimental para las tres estrategias comparadas: representación sin compresión, compresión sin pérdida basada en Blosc2 y compresión con pérdida controlada basada en ZFP.

11.1 Diseño experimental

Para Grover se consideraron circuitos con 26, 28 y 30 qubits y tres estados marcados por tamaño, uno localizado en la parte baja del espacio de búsqueda, otro en la región central y otro en la parte alta. Para QFT se evaluaron tres clases de entrada por cada tamaño de problema: estado base, superposición periódica y superposición con fase aleatoria normalizada. Las tablas presentadas a continuación reportan promedios sobre esas instancias, de modo que los resultados no dependan de un único patrón de amplitudes.

Las tres estrategias se ejecutaron sobre el mismo entorno de hardware y software, con igual número de qubits, mismo circuito, misma semilla de inicialización y misma política de recolección de métricas. En la estrategia sin compresión se utilizó la representación de referencia del simulador. En la estrategia sin pérdida se evaluó el backend basado en Blosc2 variando tamaño de bloque, tamaño de caché y nivel de compresión. En la estrategia con pérdida controlada se evaluó ZFP con la misma granularidad de bloque y con niveles de error objetivo decrecientes.

Tabla 5: Configuraciones experimentales evaluadas

Configuración	Bloque	Caché	Parámetro de compresión
B1/Z1	2^{18} amplitudes	4 entradas	Blosc2 nivel 3 / ZFP tolerancia 10^{-8}
B2/Z2	2^{19} amplitudes	8 entradas	Blosc2 nivel 5 / ZFP tolerancia 10^{-9}
B3/Z3	2^{20} amplitudes	16 entradas	Blosc2 nivel 7 / ZFP tolerancia 10^{-10}

Las métricas registradas fueron uso máximo de memoria residente, tiempo de ejecución total, razón de compresión efectiva y error máximo absoluto respecto a la simulación sin compresión. En el caso de Blosc2, al tratarse de una estrategia sin pérdida, la verificación produjo igualdad exacta en todos los estados observados; por ello, el error máximo absoluto reportado es cero en todas las configuraciones de esa estrategia.

Espacio reservado para una figura comparativa del entorno experimental y del rango de tamaños evaluados para Grover y QFT.

Figura 9: Esquema general de la campaña experimental

11.2 Resultados para Grover

Los circuitos de Grover mostraron el comportamiento más favorable para la estrategia sin pérdida basada en Blosc2. La distribución de amplitudes y la reutilización de regiones activas del vector produjeron una compresibilidad superior a la observada en QFT y permitieron que la caché amortiguara con mayor efectividad el costo de descompresión.

Tabla 6: Resultados promedio para Grover con 30 qubits

Estrategia	Config.	Mem. pico (GB)	Red. mem.	Tiempo (s)	Ratio	Error
Sin compresión	—	16.4	—	91.2	1.00	0
Blosc2	B1	10.4	36.6 %	107.8	1.58	0
Blosc2	B2	8.9	45.7 %	103.6	1.84	0
Blosc2	B3	8.6	47.6 %	110.9	1.91	0
ZFP	Z1	6.8	58.5 %	97.4	2.41	$2,1 \times 10^{-8}$
ZFP	Z2	5.9	64.0 %	95.8	2.78	$5,3 \times 10^{-8}$
ZFP	Z3	5.4	67.1 %	101.2	3.04	$1,2 \times 10^{-7}$

La configuración B2 ofreció el mejor balance para Grover dentro de la estrategia sin pérdida. Frente a la simulación sin compresión, redujo el uso máximo de memoria en 45.7 % con una sobrecarga temporal de 13.6 %. La configuración B3 produjo un ahorro de memoria ligeramente mayor, pero el incremento de tiempo fue superior debido al mayor costo de compresión y al crecimiento del tamaño de bloque.

En la comparación con ZFP, la reducción de memoria fue más agresiva y el tiempo de ejecución se mantuvo más cercano al caso sin compresión, pero a costa de introducir error numérico. Ese efecto fue especialmente visible al pasar de Z1 a Z3, donde el mayor ratio de compresión vino acompañado de un aumento progresivo del error máximo absoluto.

Espacio reservado para una gráfica de uso de memoria en Grover comparando sin compresión, Blosc2 y ZFP para las configuraciones B1/B2/B3 y Z1/Z2/Z3.

Figura 10: Uso máximo de memoria para Grover

Espacio reservado para una gráfica de tiempo de ejecución en Grover comparando las tres estrategias evaluadas.

Figura 11: Tiempo de ejecución para Grover

11.3 Resultados para QFT

Los circuitos QFT presentaron un escenario menos favorable para la compresión sin pérdida. La mayor dispersión de amplitudes redujo la regularidad local del vector de estado y con ello la efectividad de Blosc2. Aun así, la estrategia mantuvo reducciones de memoria apreciables con error nulo en todas las configuraciones evaluadas.

Tabla 7: Resultados promedio para QFT con 30 qubits

Estrategia	Config.	Mem. pico (GB)	Red. mem.	Tiempo (s)	Ratio	Error
Sin compresión	—	16.4	—	118.6	1.00	0
Blosc2	B1	11.9	27.4 %	138.2	1.38	0
Blosc2	B2	11.1	32.3 %	134.7	1.48	0
Blosc2	B3	10.8	34.1 %	140.8	1.52	0
ZFP	Z1	7.7	53.0 %	126.3	2.13	$4,9 \times 10^{-8}$
ZFP	Z2	7.0	57.3 %	124.8	2.34	$1,1 \times 10^{-7}$
ZFP	Z3	6.6	59.8 %	131.5	2.49	$2,5 \times 10^{-7}$

En QFT también se observó que la configuración B2 ofreció el mejor compromiso dentro de Blosc2. La reducción de memoria alcanzó 32.3 % con una sobrecarga temporal de 13.6 %, mientras que B3 produjo un incremento marginal en compresión con un costo de tiempo mayor. En ZFP, la reducción de memoria se mantuvo por encima de 53 % en todas las configuraciones, pero la dispersión de fases del circuito hizo crecer el error más rápido que en Grover.

Espacio reservado para una gráfica de uso de memoria en QFT comparando sin compresión, Blosc2 y ZFP.

Figura 12: Uso máximo de memoria para QFT

Espacio reservado para una gráfica de error máximo absoluto en QFT para las configuraciones Z1, Z2 y Z3.

Figura 13: Error máximo absoluto para QFT con compresión con pérdida

11.4 Comparación global de estrategias

La comparación conjunta muestra dos comportamientos claramente diferenciados. La estrategia sin pérdida basada en Blosc2 preserva exactitud total y reduce la memoria entre 32.3 % y 45.7 % en las configuraciones más equilibradas, con una sobrecarga temporal moderada cercana al 14 %. La estrategia con pérdida controlada basada en ZFP logra reducciones más agresivas, entre 57.3 % y 64.0 % en las configuraciones intermedias, pero introduce errores que dependen del circuito y del nivel de tolerancia seleccionado.

Tabla 8: Resumen comparativo de las configuraciones más favorables

Circuito	Estrategia	Red. mem.	Sobrecarga temporal	Error
Grover	Blosc2 B2	45.7 %	13.6 %	0
Grover	ZFP Z2	64.0 %	5.0 %	$5,3 \times 10^{-8}$
QFT	Blosc2 B2	32.3 %	13.6 %	0
QFT	ZFP Z2	57.3 %	5.2 %	$1,1 \times 10^{-7}$

Estos resultados muestran que la compresión sin pérdida es una alternativa efectiva cuando el objetivo principal es desplazar el límite práctico de memoria sin alterar el resultado numérico del simulador. En ese escenario, la combinación de particionamiento, compresión por bloques y caché LRU ofrece un ahorro de memoria estable con costo temporal controlado. La comparación con ZFP confirma, además, que existe un espacio adicional de reducción de memoria cuando se admite un error pequeño y acotado, aunque esa decisión corresponde a un régimen de uso distinto del perseguido por una estrategia estrictamente sin pérdida.

Espacio reservado para una figura de síntesis con memoria, tiempo y error de las configuraciones B2 y Z2 frente a la referencia sin compresión.

Figura 14: Síntesis comparativa de resultados experimentales

Referencias

- Boixo, S., Isakov, S. V., Smelyanskiy, V. N., y Neven, H. (2017). Simulation of low-depth quantum circuits as complex undirected graphical models. *arXiv preprint arXiv:1712.05384*. Descargado de <https://arxiv.org/abs/1712.05384>
- Burtscher, M., y Ratanaworabhan, P. (2009). Fpc: A high-speed compressor for double-precision floating-point data. *IEEE Transactions on Computers*, 58(1), 18–31. Descargado de <https://userweb.cs.txstate.edu/~mb92/papers/tc09.pdf> doi: 10.1109/TC.2008.131
- Chen, J., Zhang, F., Huang, C., Newman, M., y Shi, Y. (2018). *Classical simulation of intermediate-size quantum circuits*. Descargado de <https://arxiv.org/abs/1805.01450>
- Díaz, G. (2025). Memory management strategies for software quantum state simulation. *MDPI*, 7(3), 41. (Describe estrategias de memoria y referencia al prototipo TMFQS)
- Díaz, G. J. (2024). How to build a software quantum simulator. *Preprints.org*. (Incluye escenarios full-state con OpenMP, MPI y compresión)
- Díaz Toro, G. J. (2024). *Tmfqsfullstate: Prototype quantum computing simulator (full state vector)*. GitHub repository.
- Díaz, G., Steffenel, L. A., Barrios, C. J., y Couturier, J.-F. (2024). *How to build a software quantum simulator*. Preprints. Descargado de <https://doi.org/10.20944/preprints202409.1497.v1> doi: 10.20944/preprints202409.1497.v1
- Díaz Toro, G. J. (2025). *Gestión de memoria en simuladores de computación cuántica de software* (Tesis doctoral, Universidad Industrial de Santander). Descargado 2025-05-13, de <https://noesis.uis.edu.co/items/357883df-3223-45b0-9848-f7681c5b0511>
- Gheorghiu, V. (2018). Quantum++: A modern c++ quantum computing library. *PLOS ONE*.
- Gupta, N. (2025, diciembre). *Simon's algorithm: Cracking the hidden code*. Medium

- (The Quantum Ladder). Descargado de <https://medium.com/the-quantum-ladder/simons-algorithm-cracking-the-hidden-code-f55c2d9c293e> (Accedido: 2026-01-26)
- Huffman, N., Pavlichin, D., y Weissman, T. (2024). *Lossy compression for schrödinger-style quantum simulations*. Descargado de <https://arxiv.org/abs/2401.11088>
- Intel quantum simulator (intel-qs) documentation*. (s.f.). ReadTheDocs.
- Jamalidinan, S., y Cheshmi, K. (2025). *Floating-point data transformation for lossless compression*. Descargado de <https://arxiv.org/abs/2506.18062> doi: 10.48550/arXiv.2506.18062
- Jaques, S., y Häner, T. (2021). *Leveraging state sparsity for more efficient quantum simulations*. Descargado de <https://arxiv.org/abs/2105.01533>
- Li, S., Kimura, Y., Sato, H., Yu, J., y Fujita, M. (2023). *Parallelizing quantum simulation with decision diagrams*. Descargado de <https://arxiv.org/abs/2312.01570>
- Lindstrom, P., y Isenbarg, M. (2006). Fast and efficient compression of floating-point data. *IEEE Transactions on Visualization and Computer Graphics*, 12(5), 1245–1250. Descargado de <https://pubmed.ncbi.nlm.nih.gov/17080858/> doi: 10.1109/TVCG.2006.143
- Markov, I. L., y Shi, Y. (2008). Simulating quantum computation by contracting tensor networks. *SIAM Journal on Computing*, 38(3), 963–981. Descargado de <https://arxiv.org/abs/quant-ph/0511069> doi: 10.1137/050644756
- Masui, K., Amiri, M., Connor, L., Deng, M., Fandino, M., Hoefer, C., ... Vanderlinde, K. (2015). A compression scheme for radio data in high performance computing. *arXiv preprint arXiv:1503.00638*. Descargado de <https://arxiv.org/abs/1503.00638> (Describes the Bitshuffle filter and HDF5 plugin) doi: 10.48550/arXiv.1503.00638
- Miller, D. M., y Thornton, M. A. (2006). QMDD: A decision diagram structure for reversible and quantum circuits. En *Proceedings of the 36th international symposium on multiple-valued logic (ISMVL'06)* (p. 30). Descargado de <https://ieeexplore.ieee.org/>

- document/1623982/ doi: 10.1109/ISMVL.2006.35
- Pednault, E., Gunnels, J. A., Nannicini, G., Horesh, L., Magerlein, T., Solomonik, E., ... Wisnieff, R. (2020). *Pareto-efficient quantum circuit simulation using tensor contraction deferral*. Descargado de <https://arxiv.org/abs/1710.05867>
- PennyLane. (2023). *Grover's algorithm / pennylane demos*. Web tutorial. Descargado de https://pennylane.ai/qml/demos/tutorial_grovers_algorithm (Accedido: 2026-01-26)
- qsim cirq interface documentation*. (s.f.). Quantum AI documentation.
- qsim: Fast c++ and python library for state-vector simulation of quantum circuits*. (s.f.). GitHub repository.
- The quantum exact simulation toolkit (quest) documentation*. (s.f.). Project documentation website.
- Quest: Quantum exact simulation toolkit*. (s.f.). GitHub repository.
- Song, Y., Li, Z., y Wu, J. (2023). *Mera: Memory reduction and acceleration for quantum circuit simulation via redundancy exploration*. Descargado de <https://arxiv.org/abs/2411.15332>
- Viamontes, G. F., Markov, I. L., y Hayes, J. P. (2005). Graph-based simulation of quantum computation in the density matrix representation. *Quantum Information & Computation*, 5(2), 113–130. Descargado de <https://arxiv.org/abs/quant-ph/0403114>
- Vidal, G. (2003). Efficient classical simulation of slightly entangled quantum computations. *Phys. Rev. Lett.*, 91(14), 147902. doi: 10.1103/PhysRevLett.91.147902
- Vinkhuijzen, L., Coopmans, T., Elkouss, D., Dunjko, V., y Laarman, A. (2023, septiembre). LIMDD: A Decision Diagram for Simulation of Quantum Computing Including Stabilizer States. *Quantum*, 7, 1108. Descargado de <https://quantum-journal.org/papers/q-2023-09-11-1108/> doi: 10.22331/q-2023-09-11-1108
- Wu, X.-C., Di, S., Cappello, F., Finkel, H., Alexeev, Y., y Chong, F. T. (2018). Amplitude-aware lossy compression for quantum circuit simulation. En *Proceedings of the 4th in-*

- ternational workshop on data reduction for big scientific data (drbsd-4) at sc18*. IEEE.
- Wu, X.-C., Di, S., Dasgupta, E. M., Cappello, F., Finkel, H., Alexeev, Y., y Chong, F. T. (2019a). Full-state quantum circuit simulation by using data compression. En *Proceedings of the international conference for high performance computing, networking, storage and analysis (sc'19)*. IEEE/ACM. doi: 10.1145/3295500.3356155
- Wu, X.-C., Di, S., Dasgupta, E. M., Cappello, F., Finkel, H., Alexeev, Y., y Chong, F. T. (2019b). Full-state quantum circuit simulation by using data compression. *arXiv*. Descargado de <https://arxiv.org/abs/1911.04034> (Accedido: 2026-01-26)
- Wu, X.-C., Di, S., y cols. (2018). Amplitude-aware lossy compression for quantum circuit simulation. En *Sc18 workshops (drbsd)*. Descargado de https://sc18.supercomputing.org/proceedings/workshops/workshop_files/ws_drbsd112s1-file1.pdf (Accedido: 2026-01-26)
- Zhang, B., Fang, B., Ye, F., Gu, Y., Tallent, N., Tan, G., y Tao, D. (2024a). *Overcoming memory constraints in quantum circuit simulation with a high-fidelity compression framework*. Descargado de <https://arxiv.org/abs/2410.14088>
- Zhang, B., Fang, B., Ye, F., Gu, Y., Tallent, N., Tan, G., y Tao, D. (2024b). *Overcoming memory constraints in quantum circuit simulation with a high-fidelity compression framework*. Descargado de <https://arxiv.org/abs/2410.14088>